

Joule Ecosystem

Submitted by

Daniele Famà

daniele.fama@telecom-paris.fr



MultiMedia Lab

Image, Data and Signal Department

Laboratoire de Traitement et de Communication de l'Information (LTCI)

Télécom Paris, Institut Polytechnique de Paris

Summer Internship 2026

Abstract

This report presents a complete pipeline to measure and optimize the energy consumption of neural network inference on edge devices. It introduces three software components:

JouleQuest measures the power of AI workloads on embedded boards (Jetson Nano, Jetson AGX Orin, Raspberry Pi 5) using an INA226EVM sensor. It builds energy lookup tables for common neural network layers by automating calibration, clock synchronization, and workload sizing.

JouleGrad turns these tables into differentiable energy estimates, allowing us to use them as penalties during neural network training.

JouleNAS uses these estimates to perform an energy-aware search for the best channels in frozen networks (SimpleConv and ResNet-18 on CIFAR-10 and Imagenette). This finds architectures specifically optimized for the energy cost of the target board.

Part of this report serves as a practical guide for future developers to extend the project and collect new measurements.

Contents

Introduction	1
1 JouleQuest: Measurement and data-acquisition layer	2
1.1 Hardware configuration	3
1.2 Software configuration	3
1.2.1 The two-host setup	5
1.2.2 SSH configuration	5
1.3 How it works, technically	6
1.3.1 One experiment, start to finish	6
1.3.2 Why the bursts are “adaptive”	7
1.3.3 Clock synchronization	8
1.3.4 Manifest structure and quality flags	8
1.4 Running the measurements	10
1.4.1 Smoke test: prove everything works in five minutes	10
1.4.2 Full suite: the campaign launcher	11
1.4.3 Current measurement setup and campaign parameters	12
1.4.4 What a full campaign actually measures	13
1.4.5 Manual fallback	16
1.5 Processing the results	16
1.6 [Future work] Does the energy of a network equal the sum of its parts?	17
1.6.1 Open directions	18
1.7 Tests and troubleshooting	19
2 JouleGrad: Differentiable energy-estimation layer	20
2.1 Improvements over energy_estimator	20
2.2 Handling out-of-range queries	21
2.2.1 The problem with silent extrapolation	21
2.2.2 JouleGrad policies	21
2.3 Performance architecture	22
2.3.1 Loading grids into PyTorch tensors	22
2.3.2 Automatic GPU caching	22
2.3.3 Vectorized interpolation	22
2.3.4 Batching layer evaluations	23
2.3.5 Preparing the model structure	23
2.3.6 Fast loss calculation vs. detailed diagnostics	23
2.3.7 Floating-point precision	23
2.4 Usage	23
2.4.1 Layer-level queries	24
2.4.2 Model-level queries	24

3	JouleNAS: Environmentally-aware optimization layer	25
3.1	How the mask optimization works	26
3.2	How the energy term is computed	26
3.2.1	Discrete versus continuous energy modes	26
3.2.2	From effective sizes to energy: multilinear interpolation	27
3.2.3	Why gradients flow to the mask logits	27
3.3	Running an experiment	28
3.3.1	Optimization controls and hyperparameters	28
3.4	Experimental results and what they tell us	29
3.4.1	Embedded GPU target: NVIDIA Jetson AGX Orin	29
3.4.2	Cross-hardware generalization: evaluation on Raspberry Pi 5	34
3.4.3	Amortizing GPU execution overhead: ResNet-18 pruning at batch size 32	36
4	The Bridge: from measurements to decisions	39
4.1	The pipeline	39
4.2	From masks to physical deployment: validating with JouleQuest	39
4.2.1	Selecting the target models: $\lambda_E = 0.005$ discrete	39
4.2.2	Reconstructing the hardware-specific pruned architectures	40
4.2.3	Handling residual shortcuts and BatchNorm	40
4.2.4	Automated testing and validation in JouleQuest	41
4.2.5	Physical hardware validation on NVIDIA Jetson AGX Orin (batch size 32)	42
4.3	Where the chain is weakest	43
5	Future Work	45
5.1	Measurement layer (JouleQuest)	45
5.2	Search layer (JouleNAS)	45
6	Conclusion	47
	Acknowledgements	48
	References	49

Introduction

This document is both an internship report and a practical guide. It explains what we built, how the pieces fit together, and how to take new energy measurements. You do not need prior knowledge of the project to follow along. Every chapter starts with a quick test to make sure things work, followed by the commands to run the full suite. Since working with hardware can be tricky, the text warns you about common issues in advance.

The big picture

This project studies the energy consumption of AI models on edge devices (like the Raspberry Pi 5 and Jetson boards) and uses this data to guide Neural Architecture Search. The work is split across three main tools:

- **JouleQuest (Measurement)**: Runs tests on a target board and records power draw using an INA226EVM sensor. A single command gives you clean data in CSV and JSON formats.
- **JouleGrad (Estimation)**: Turns JouleQuest measurements into differentiable energy estimates for common layers (like `Linear` and `Conv2d`). This allows us to use energy consumption as a penalty during training.
- **JouleNAS (Search)**: A Neural Architecture Search pipeline that uses JouleGrad's estimates to find network architectures that are both accurate and energy-efficient.

A final chapter connects everything: measure with JouleQuest, build the lookup table, estimate with JouleGrad, and search with JouleNAS.

How to read this document

Each chapter is self-contained, so you can jump directly to the tool you need. If you need to take new measurements, start with the JouleQuest chapter. It covers wiring, software setup, quick tests, and the main campaign commands.

A quick disclaimer: hardware setups are fragile and each board behaves slightly differently. If a command fails, check the wiring and troubleshooting notes first. Configuration issues are much more common than software bugs.

Chapter 1

JouleQuest: Measurement and data-acquisition layer

This chapter introduces **JouleQuest**, the automated measurement tool. JouleQuest runs AI models (like single Conv2d layers or full ResNet18 networks) on a target board and measures their power consumption using an INA226EVM sensor. Once you configure the hardware and software, a single command runs the entire experiment.

The official documentation in the repository contains the full details. The `README.md` is the main reference for commands, and the `docs_new/` directory provides detailed guides:

- `experiment_campaign.md` — the complete campaign matrix: which layer families, widths, boards, and operating conditions were measured, and why those measurements were chosen.
- `adaptive_burst_measurement.md` — the dynamic inferences-per-burst protocol, explaining how the acquisition logic keeps each energy sample stable, repeatable, and within the intended time budgets (recommended reading, section 1.3.2).
- `joulegrad_energy_estimation.md` — the bridge between raw measurements and the lookup table used downstream by JouleGrad, showing how the data are transformed into a usable estimator input.
- `cli_acquisition_report.md` — explains how to run measurements directly from the Linux terminal instead of using the graphical interface, testing communication speed and commands with the sensor.
- `energy_lookup_table_measurement_audit.md` — checks that the final lookup tables accurately match the real measurements taken from the board.
- `run_manager_update.md` — explains the changes and new features added to `run_manager.py` compared to the older version of JouleQuest.
- `terminal_acquisition_full_report.md` — provides detailed internal documentation on the terminal acquisition architecture, setup prerequisites, and sequence of steps for unattended or headless measurement campaigns.
- `pruned_resnet_measurement.md` — documents the spec-driven workflow for pruned ResNet-18 composition measurements, detailing how a single JouleNAS pruning log drives dynamic model construction, component layer scheduling, deterministic architecture hashing, and physical validation.

This chapter summarizes the full workflow and adds the context that only makes sense when the measurement protocol and the data processing are considered together.



(a) Front view.



(b) Rear label (5 V, 5 A output rating).

Figure 1.1: Low-power supply used for the Raspberry Pi 5 and Jetson Nano.

1.1 Hardware configuration

This section extends the hardware setup described in previous work. Depending on the board's power requirements, we used two configurations:

- **Low power** (Figure 1.1): Raspberry PI5, Jetson Nano. The supplier already has the black and red connectors ready to be inserted in the board, so no extra cabling is required.
- **High power** (Figure 1.2): Jetson Nano Orin, Jetson AGX Orin. The supplier needs a cable extension to work (Figure 1.2c).

A few things worth double-checking before you power anything on:

- Know your shunt resistor value and the maximum current your board can draw: they become `-shunt-ohms` and `-max-expected-current-a`.
- The TI-SCB is the USB box that talks to the INA226EVM and exposes a serial port on the PC. With exactly one plugged in, the software finds it by itself; with more than one of the same family, pass the port explicitly.

Once all the cables are set up you can move on to the software configuration.

1.2 Software configuration

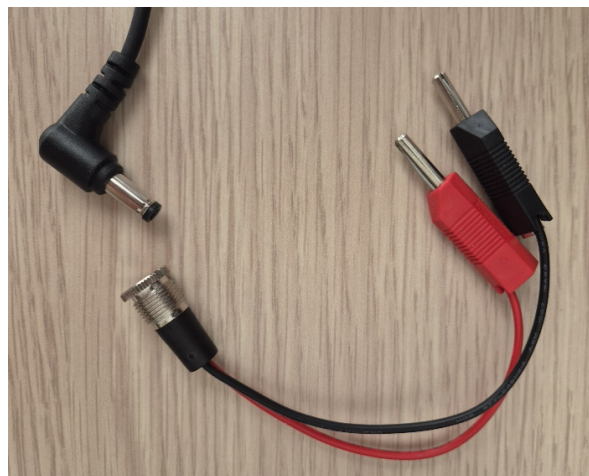
You do *not* need the full repository on the board. JouleQuest uses a two-host setup, and each side only needs a handful of files.



(a) Front view.



(b) Rear label (19 V, 2.37 A output rating).



(c) DC cable extension adapter.

Figure 1.2: High-power supply used for the Jetson Orin Nano and Jetson AGX Orin, including the required cable adapter.

1.2.1 The two-host setup

Two machines are involved:

- **The PC** is connected to the TI-SCB over USB. It runs the orchestrator (`automated_measurement.py`) and the logger (`ina226_serial_logger.py`), which writes the raw INA226 samples to CSV. Python and `pyserial` required.
- **The board** (Jetson, Raspberry, ...) runs the model. It needs three files — `run_manager.py`, `runner.py`, `base_runner.py` and the `layers` directory for more advanced experiments (e.g. resnet, attention).

To prepare a *new* board, copy the inference-side scripts over SSH:

```
scp -r run_manager.py runner.py base_runner.py layers\  
-o ProxyJump=JUMP_USER@JUMP_HOST \  
BENCH_USER@INFERENCE_HOST:/path/to/JouleQuest/
```

On the PC side, clone the repository and install the acquisition dependencies (the logger needs `pyserial`; the result-processing scripts need `pandas`, `scikit-image` and `matplotlib`):

```
python -m pip install -r Docs/INA226EVM/requirements.txt  
python -m pip install pandas scikit-image matplotlib
```

1.2.2 SSH configuration

SSH is the most common failure point of the whole setup: **if SSH asks for a password, the run fails**, because the automation runs with `BatchMode=yes` and cannot type one. Set up key based authentication *before* launching anything — the quickest way is `ssh-copy-id` — and remember also to authorize the jump host, if your board is reached through one.

The last piece of configuration is a small JSON file that tells the PC where the board is: `measurement_hosts.local.json`. You should start editing it with any text editor:

```
cp measurement_hosts.example.json measurement_hosts.local.json  
nano measurement_hosts.local.json
```

This is all there is inside — just replace the placeholders with the real hostnames, user names and paths:

```
{  
  "runner_host": "BENCH_USER@INFERENCE_HOST",  
  "jump_host": "JUMP_USER@JUMP_HOST",  
  "remote_directory": "/path/to/joulequest",  
  "remote_python": "/path/to/python",  
  "remote_manifest_directory": "measurement_manifests",  
  "ssh_connect_timeout_s": 10.0,  
  "ssh_options": []  
}
```

To give you a concrete idea, this is a real configuration used for the Raspberry Pi 5 measurements (addresses anonymized for this report; the original file stays on the measurement host PC):

```
{  
  "runner_host": "pi@<BOARD_IP>",  
  "jump_host": "<USER>@<JUMP_HOST>",  
  "remote_directory": "/home/pi/energybanera",  
  "remote_python": "/home/pi/miniconda3/bin/python",  
  "remote_manifest_directory": "measurements_pi5",  
  "ssh_connect_timeout_s": 10.0,  
  "ssh_options": []  
}
```

In words: `runner_host` is the board under test, `jump_host` is the gateway used to reach it (leave it empty if the board is directly reachable), `remote_directory` is where the runner scripts live on the board, and `remote_python` is the Python interpreter on the board — point it at the environment with PyTorch installed. The remaining fields should not need changes.

Since it contains the addresses of the team’s machines, keep it private: it is already listed in `.gitignore`, so it will not end up in Git by mistake.

Once the file is in place, run this command:

```
ssh -J JUMP_USER@JUMP_HOST \  
-o BatchMode=yes \  
BENCH_USER@INFERENCE_HOST \  
'printf "SSH_OK: "; hostname'
```

If the output is `SSH_OK:` followed by the board’s hostname, the configuration is complete and you can move on to running measurements.

1.3 How it works, technically

You *can* skip this section and still use JouleQuest — but you shouldn’t. A quick read will help you understand the files you get back and spot unusual results, it also explains the reasoning behind some parameter choices, like the sampling rate. Nothing here requires action from you: everything described below happens automatically.

1.3.1 One experiment, start to finish

Each experiment is driven by `automated_measurement.py` on the PC, and goes through the same steps every time:

1. The PC starts the experiment on the board over SSH.
2. The board runs a **warm-up** (not measured): the first forward pass is slower than the others, because the GPU and the libraries need a moment to get ready, so one burst is executed and deliberately thrown away.
3. The board runs a short **calibration** (not measured): it times a few small batches to learn how long one inference takes, and from that it computes how many inferences are needed to make each measured burst long enough.
4. The board tells the PC to **start recording**: the INA226 logger begins writing power samples to a CSV file.
5. The **measured cycles** run: a few bursts with idle pauses in between, plus some idle time before and after, so the baseline consumption can be subtracted later.
6. The board tells the PC to **stop recording**. The logger closes the file *before* the board writes its report, so this final bookkeeping never pollutes the measurement.
7. The PC downloads the report (called *manifest*), saves it next to the CSV, and deletes the copy on the board to save space (use `-keep-remote-manifest` if you want to keep it for debugging).

Each experiment leaves exactly two files behind:

```
measurements/runs/<campaign_id>.csv    # raw INA226 samples  
measurements/runs/<campaign_id>.json    # the manifest
```

The manifest is the experiment’s report card: what was measured, how long each burst lasted, the real sampling rate, and a `quality_status` flag. The command’s exit code sums everything up: 0 all good, 2 the model ran but the recording is invalid, 1 the experiment failed, 130 you stopped it with `Ctrl+C` (and everything was cleaned up properly).

Furthermore, to guarantee rigorous benchmarking on multi-core CPU targets (such as the Raspberry Pi 5), the execution harness supports an explicit `CPU_THREADS` variable forwarded by `run_measurement_campaign.sh` over SSH. The execution environment—including configured threads, active PyTorch thread count, and available cores—is certified directly in the manifest under `runner_environment`. Because this metadata is recorded during the initial setup phase outside the measurement window, it introduces zero I/O or CPU overhead during the active inference bursts.

1.3.2 Why the bursts are “adaptive”

Older versions of this project ran the same huge number of inferences for every layer. That worked, but it wasted a lot of time on slow layers while being too short to be meaningful for fast ones. Now the length of each burst is computed automatically, so that every measured region is long enough both in *time* and in *number of samples*:

$$T_{\text{required}} = m \cdot \max\left(T_{\text{target}}, \frac{N_{\text{samples,min}}}{f_s}\right),$$

where:

- f_s is how many samples per second the INA226 takes (`-sampling_rate_hz`);
- $N_{\text{samples,min}}$ is the minimum number of samples you want inside each active region (`-min_active_samples`);
- m is a safety margin (`-burst_duration_margin`, default 1.2, i.e. 20% longer than strictly needed).

In words: the burst must last at least `-target_burst_seconds`, *and* it must contain enough samples for the statistics to be meaningful — whichever of the two asks for more time wins, plus the safety margin. For my experiments I have not used a target in seconds but in a minimum amount of active samples = 100.

One practical note on the sampling rate. The INA226 can, in principle, go above the 100 Hz we use in the campaigns, so it is natural to ask why we are limiting it to 100 Hz. The reason is the communication overhead: every sample requires the PC to talk to the sensor through the TI-SCB and read several internal registers, and that back-and-forth takes time. When we pushed the requested rate higher, the transfer could not keep up and samples were actually *lost* — the achieved rate became irregular and lower than what we obtained by simply asking for 100 Hz. This is not a theoretical limit: we observed it empirically by comparing requested versus achieved sampling rates, and 100 Hz turned out to be the fastest setting at which no samples go missing. The achieved rate is recorded in every manifest, so you can always check that your acquisition behaved.

Once calibration has measured how long one inference takes ($t_{\text{inference}}$), the number of inferences per burst is just $\lceil T_{\text{required}}/t_{\text{inference}} \rceil$. This estimate is double-checked before the real measurement: the runner performs a few extra *validation* bursts (not measured) with exactly that count and verifies that even the shortest one is long enough. If not, the count is raised automatically and checked again for at most `-calibration-sizing-max-attempts` times.

1.3.3 Clock synchronization

There is one subtlety worth knowing about. The power samples are timestamped by the PC, but the bursts run on the board — and the two clocks do not agree perfectly. To match each burst with the right samples, the two machines compare their clocks right before and after the recording, and the manifest stores the translated time windows.

If the two clocks can be aligned precisely enough, processing uses those windows directly. If not (for example the network was slow during the comparison), nothing is lost: the processing script finds the active regions on its own, by looking at where the power trace rises above the baseline (Otsu thresholding plus hysteresis, a standard image-segmentation technique applied to the trace). The manifest records which method was used, so you can always check.

1.3.4 Manifest structure and quality flags

Every completed experiment leaves behind two files: a raw CSV trace of INA226 electrical readings and a companion JSON audit called the *manifest* (using `schema_version: 2`). While the CSV stores timestamps, voltages, and currents, the manifest records the complete execution history, including workload policies, latency calibration, clock synchronization, cycle-level timing boundaries, and data-quality assessments.

Listing 1.1 shows the structure of a representative manifest produced by JouleQuest:

Listing 1.1: Representative structure of a JouleQuest manifest JSON (`schema_version: 2`).

```
{
  "schema_version": 2,
  "campaign_id": "20260813T215234Z_ResNet18_224_1000_bs1_b901e0d5",
  "status": "COMPLETE",
  "quality_status": "OK",
  "quality_flags": [],
  "model_path": "Models/CUDA/ResNet18/ResNet18_224_1000.pt",
  "backend": "cuda",
  "input_batch_size": 1,
  "calibration": {
    "target_seconds": 1.0,
    "repetitions": 8,
    "estimated_latency_seconds": 0.00535,
    "relative_mad": 0.0041,
    "coefficient_of_variation": 0.0052,
    "quality_flags": []
  },
  "burst_validation": {
    "rounds": 1,
    "passed": true,
    "validation_min_seconds": 1.218,
    "validation_median_seconds": 1.221
  },
  "acquisition": {
    "requested_sampling_rate_hz": 100.0,
    "achieved_sampling_rate_hz": 99.9998,
    "sample_count": 47443,
    "clock_alignment": {
      "status": "COMPLETE",
      "uncertainty_seconds": 0.0021,
      "classification_eligible": true
    }
  },
  "measurement": {
    "total_executed_inferences": 22800,
    "cycles": [
      {
```

```

    "cycle": 1,
    "requested_inferences": 228,
    "executed_inferences": 228,
    "elapsed_seconds": 1.220,
    "estimated_ina226_samples": 122.0,
    "quality_flags": [],
    "start_event": {
      "monotonic_seconds": 1542.31,
      "aligned_elapsed_seconds": 5.02
    },
    "end_event": {
      "monotonic_seconds": 1543.53,
      "aligned_elapsed_seconds": 6.24
    }
  }
]
}
}

```

Where quality flags are located

Quality metrics are tracked across multiple levels within the manifest hierarchy:

- **Cycle-level quality flags** (`cycles[i].quality_flags`): Each measured cycle tracks its own local flags. If a specific burst suffers from anomalous duration or sample starvation, the diagnostic flag is recorded directly in that cycle's entry.
- **Calibration quality flags** (`calibration.quality_flags`): Records stability or convergence anomalies encountered while determining single-inference latency prior to acquisition.
- **Global status and aggregate flags**: Located at the root of the JSON object, `quality_flags` gathers the deduplicated union of all flags emitted across the entire run. If this list is empty, `quality_status` is assigned "OK". If one or more flags are present anywhere, `quality_status` becomes "REVIEW".

Causes of the REVIEW flag: undersampling and duration anomalies

The REVIEW status indicates that an experiment ran to completion (`status: COMPLETE`) but experienced timing or resolution discrepancies that warrant inspection:

1. **Undersampling (UNDER_RESOLVED)**: This flag is triggered for a cycle when the number of actual power samples captured during the forward pass falls below the required threshold:

$$N_{\text{actual}} = t_{\text{elapsed}} \cdot f_s < N_{\text{samples,min}}.$$

With our default parameters ($f_s = 100$ Hz and $N_{\text{samples,min}} = 100$), an active burst must last at least 1.0 s. If a burst finishes in less than one second, the cycle captures fewer than 100 samples. This occurs when an operation executes faster than calibration predicted, leaving too few discrete measurement points to separate the steady-state plateau from transient edge effects.

2. **Severe duration anomaly (DURATION_ANOMALY)**: A more severe timing distortion flag is raised when the actual measured duration deviates by more than a factor of two from the calibration estimate:

$$\frac{t_{\text{elapsed}}}{t_{\text{estimated}}} < 0.5 \quad \text{or} \quad \frac{t_{\text{elapsed}}}{t_{\text{estimated}}} > 2.0.$$

This condition captures two distinct anomalies:

- *Severe undersampling* ($< 0.5\times$): The execution completed in less than half the expected time. This severely depresses the sample count and usually triggers both `DURATION_ANOMALY` and `UNDER_RESOLVED`.
- *Heavy oversampling and latency dilation* ($> 2.0\times$): The execution took more than double the expected time. This can occur when the device experiences unexpected latency spikes due to memory bus contention, GPU thermal throttling, or CPU background scheduling.

3. **Calibration instability flags:** A `REVIEW` can also stem from pre-acquisition calibration: `CALIBRATION_UNSTABLE` is raised when the latency relative MAD or coefficient of variation exceeds 15%, and `CALIBRATION_SIZING_NOT_CONVERGED` indicates that the pilot failed to reach the 1.0s target duration within 5 attempts.

Why oversampled runs were treated as valid

A crucial design choice was made regarding runs flagged for `REVIEW` due to oversampling. When generating downstream energy lookup tables (`build_energy_lookup_table.py`), we explicitly accept experiments whose quality status is either `"OK"` or `"REVIEW"`:

```
data = data[data["quality_status"].isin(["OK", "REVIEW"])]
```

This decision is physically grounded: **heavy oversampling does not harm energy estimation, whereas undersampling does**. When a burst runs longer than planned, the INA226 simply captures *more* discrete power readings across the execution plateau ($N_{\text{actual}} \gg 100$). Because energy is calculated by numerical integration over time ($E = \int [P(t) - P_{\text{idle}}] dt$), having a denser set of discrete samples across the plateau improves or preserves the numerical precision of the Riemann sum without introducing systematic bias.

In contrast, severe undersampling risks missing transient features and provides insufficient points to compute a clean trimmed mean. By accepting oversampled experiments marked as `REVIEW`, we retain high-fidelity measurements that would otherwise be discarded unnecessarily, preserving complete coverage in our lookup tables.

1.4 Running the measurements

1.4.1 Smoke test: prove everything works in five minutes

Before launching anything long, run this minimal experiment. It touches the whole chain (SSH, inference, recording, file transfer) with a small model, and finishes in a few minutes:

```
python automated_measurement.py \
--connection-config measurement_hosts.local.json \
--backend cuda \
--output-directory measurements/runs \
--shunt-ohms 0.012 \
--max-expected-current-a 5.0 \
--number_of_cycles 2 \
--sleep_time 3 \
--target_burst_seconds 2 \
--model Models/CUDA/Linear/Linear_64_64.pt
```

So what does each argument mean? Here is the smoke test, decoded:

- `-connection-config`: the JSON file from the SSH section — who the board is and how to reach it.
- `-backend`: where the model runs on the board (`cuda`, `cpu`, or `tpu`). Raspberry PIs run only on `cpu` mode.

- `-output-directory`: where the CSV/JSON pair is saved on the PC.
- `-shunt-ohms` / `-max-expected-current-a`: the electrical settings of your measurement chain (the two numbers from the hardware section).
- `-number_of_cycles`: how many bursts are measured (default 5; we use 2 here just to be quick).
- `-sleep_time`: idle seconds between bursts (default 10).
- `-target_burst_seconds`: how long each burst should last (2 s here instead of the default 10, again just to be quick).

These are only the most common ones. The full list — warm-up, calibration, validation, clock sync, idle windows and all the defaults — is documented in the CLI parameter reference table of `docs_new/adaptive_burst_measurement.md` (Section 9), and every value is also recorded in the manifest of each experiment.

One thing that can confuse at first: `-model` is a path *on the board*, while `-output-directory` is *on the PC*. And “path” is maybe not so accurate as word: a full file path is only needed if you actually have a checkpoint to load. The notation above (`Models/CUDA/Linear/Linear_64_64.pt`) is enough — if the file does not exist, the runner simply reads the base path (`Linear_64_64.pt`), extracts the architecture and dimensions from the name, and starts from randomly initialized parameters. This is the default configuration, and it is exactly what you want for energy measurements.

With exactly one TI-SCB connected you can omit `-port`; otherwise find the device path with `ls /dev/serial/by-id/` and pass it explicitly.

If the command exits with code 0 and a fresh CSV/JSON pair appears in the output directory, the setup works and you can move to the real thing.

1.4.2 Full suite: the campaign launcher

One experiment at a time is nice for testing, but measuring the complete layer matrix (Linear, Conv, attention, and friends) by hand would take forever. Before showing the command, one design choice is worth explaining, because it is deliberate: `automated_measurement.py` runs *exactly one* experiment and nothing more. Scheduling the whole campaign is left to a separate shell script, `run_measurement_campaign.sh`, which simply calls the Python script once per model. Keeping the two roles separated means the Python code stays focused on doing one experiment well (SSH, calibration, recording, manifest), and you remain free to run any single configuration by hand — like the smoke test above, or any custom parameter set you need — without touching the campaign machinery. The launcher is only a convenience on top:

```
./run_measurement_campaign.sh --board BOARD_LABEL --suite all
```

Before launching it for real, you can see every command it would run — without starting anything:

```
./run_measurement_campaign.sh --board BOARD_LABEL --suite all --dry-run
```

Things worth knowing:

- One invocation measures exactly one board. When it finishes, power down, swap the board, update the connection file and the current range, and relaunch with a new `BOARD_LABEL`.
- Results go to `measurements/runs/BOARD_LABEL/`. Experiments that already have a COMPLETE manifest are skipped, so `Ctrl+C` is safe: relaunch with the same board label and the campaign resumes where it stopped.

Category	Parameters and Values	Operational Role and Physical Rationale
Workload cycles	NUMBER_OF_CYCLES = 100 BATCH_SIZE = 1	100 measured repetitions ensure high statistical significance, averaging out scheduling jitter and power rail fluctuations.
Burst duration	SAMPLING_RATE_HZ = 100 MIN_ACTIVE_SAMPLES = 100 BURST_DURATION_MARGIN = 1.2	100 Hz avoids USB/serial congestion; a minimum of 100 samples guarantees $T_{\text{burst}} \geq 1.0\text{ s}$, with a 20% safety margin ($m = 1.2$).
Warm-up & Rest	WARMUP_INFERENCES = 3 SLEEP_TIME = 3 s EXPERIMENT_COOLDOWN = 10 s	3 unmeasured passes prime CUDA kernels and memory allocators; 3 s inter-burst sleep and 10 s inter-run pause prevent cumulative thermal drift.
Calibration	CALIBRATION_TARGET = 1.0 s CALIBRATION_REPETITIONS = 8 MAX_RELATIVE_MAD = 0.15	Measures median latency across 7 retained runs (first discarded); rejects calibration if relative MAD or coefficient of variation exceeds 15%.
Validation gate	VALIDATION_REPETITIONS = 3 VALIDATION_SAFETY_MARGIN = 1.1	3 unmeasured verification bursts confirm duration before recording; scales count if execution runs faster than expected (up to 3 rounds).
Idle baselines	LEADING_IDLE = 5 s TRAILING_IDLE = 30 s SAFETY_MARGIN = 2 s	Captures pre- and post-workload quiet windows, enabling accurate subtraction of baseline board static consumption (P_{idle}).
Electrical limits	SHUNT_OHMS = 0.012 Ω MAX_EXPECTED_CURRENT = 5.0 A	Hardware sense resistor value (12 m Ω) and full-scale current limit programmed into the INA226 calibration register.

Table 1.1: Key configuration parameters in `run_measurement_campaign.sh` governing the automated acquisition protocol.

- It stops at the first error by default. If you prefer to collect failures and fix them later, use `-continue-on-error`.
- All parameters (cycles, sampling rate, model matrix, ...) are variables at the top of the script, and can be overridden with environment variables — no need to edit the script itself.
- If the board tends to heat up, add a pause between experiments with `-experiment-cooldown-seconds 60`.

The complete campaign matrix (which Linear, Conv and attention configurations are measured, and with which parameters) is documented in `docs_new/experiment_campaign.md`.

1.4.3 Current measurement setup and campaign parameters

The full execution parameters for each campaign are orchestrated by `run_measurement_campaign.sh` and enforced on the target board by `run_manager.py`. To guarantee that measurements across different boards (Raspberry Pi 5, Jetson Nano, Jetson Orin Nano, Jetson AGX Orin) are reproducible, thermally stable, and statistically reliable, the campaign enforces a standardized set of operational hyperparameters.

Table 1.1 details the essential configuration parameters governing each measurement run: A few characteristics of this setup warrant particular emphasis:

- **Statistical robustness through 100 cycles:** Running 100 cycles per layer produces 100 independent active-region energy measurements. This large sample count dampens operating-system noise, thread scheduling differences, and background memory bus activity, providing narrow confidence intervals.
- **Strict exclusion of setup overhead:** The 3 warm-up inferences and all calibration sizing bursts run strictly *before* the INA226 acquisition command is issued. As a result, model initialization and kernel compilation overhead never contaminate the measured CSV trace.
- **Thermal discipline:** Because energy consumption drifts as silicon temperature rises, maintaining thermal equilibrium is critical. The 3-second inter-cycle sleep, the 30-second trailing idle, and the 10-second inter-experiment cooldown ensure that boards dissipate heat steadily and do not enter thermal throttling regimes during the campaign.

1.4.4 What a full campaign actually measures

The launcher groups the experiments into *suites*, selected with `-suite` (comma-separated, or `all`). The available suites are listed below (see Table 1.2 for the full composition of `all`):

- **linear** (121 experiments): all input/output pairs over the sizes 64–8192, plus the low-feature points 1, 8 and 32 added for the pruning-oriented coverage (the combined 11×11 grid);
- **conv** (1648 experiments): the generic Conv matrix, after removing the channel/image-size combinations deliberately skipped by the launcher (see table 1.4), plus two pruning extensions — a complete 3×3 /pad-1 grid at spatial size 2 and a complete 1×1 pointwise grid over log-spaced channels;
- **attention** (105 experiments): sequence lengths 128–2048 $\times$ embedding dimensions 128–4096 $\times$ head dimensions 32, 64, 128, always at batch 1;
- **rotaryattention** (35 experiments): the same sequence and embedding axes for the rotary variant, with the head dimension fixed at 64;
- **lenet** (12 experiments): the complete LeNet model plus its 11 standalone operations — the material for the sum-of-parts analysis of Section 1.6;
- **resnet18** (30 experiments): the complete ResNet-18 plus one experiment for each distinct operation shape in it (convolutions,
- **resnet50** (54 experiments): the same decomposition for the bottleneck-based ResNet-50;
- **pruned_resnet18** (dynamic count, e.g. 23 experiments): a spec-driven validation suite that schedules the dense baseline, the runtime-configured pruned ResNet-18 instantiated from a JouleNAS pruning JSON log, and all its constituent **Conv2d** and **Linear** operations for physical composability verification (detailed in Chapter 4.2).

Linear experiment matrix (Table 1.3). The Linear suite benchmarks fully-connected layers across a wide range of feature dimensions. In Table 1.3:

- **Rows represent the input feature dimension** (d_{in}).
- **Columns represent the output feature dimension** (d_{out}).

Suite	Experiments	Included by all
Linear	121	yes
Conv	1648	yes
Attention	105	yes
Rotary attention	35	yes
LeNet + components	12	yes
Default all total	1921	—
ResNet-18 + components	30	no
ResNet-50 + components	54	no
All suites together	2005	—

Table 1.2: Number of experiments scheduled by each campaign suite with the default axes. **all** includes only the basic layer suites and LeNet; the ResNet suites must be requested explicitly.

$d_{\text{in}} \backslash d_{\text{out}}$	64	128	256	512	1024	2048	4096	8192	1	8	32
64	●	●	●	●	●	●	●	●	○	○	○
128	●	●	●	●	●	●	●	●	○	○	○
256	●	●	●	●	●	●	●	●	○	○	○
512	●	●	●	●	●	●	●	●	○	○	○
1024	●	●	●	●	●	●	●	●	○	○	○
2048	●	●	●	●	●	●	●	●	○	○	○
4096	●	●	●	●	●	●	●	●	○	○	○
8192	●	●	●	●	●	●	●	●	○	○	○
1	○	○	○	○	○	○	○	○	○	○	○
8	○	○	○	○	○	○	○	○	○	○	○
32	○	○	○	○	○	○	○	○	○	○	○

Table 1.3: Linear experiment matrix. Rows represent input feature dimensions (d_{in}) and columns represent output feature dimensions (d_{out}). Filled bullets (●) are the baseline 8×8 grid; open circles (○) are the 57 extra pruning points added by $\{1, 8, 32\}$, yielding $64 + 57 = 121$ Linear experiments.

The baseline matrix comprises the symmetric Cartesian product over eight powers of two: $d_{\text{in}}, d_{\text{out}} \in \{64, 128, 256, 512, 1024, 2048, 4096, 8192\}$. These 64 standard configurations are shown as filled bullets (●).

When applying neural architecture search or channel pruning, layers are frequently trimmed down to very narrow widths. To ensure the surrogate energy model can interpolate accurately without large extrapolation errors near zero, the launcher introduces three low-feature knots: $\{1, 8, 32\}$ along both axes. The open circles (○) represent these extra pruning points, scheduled whenever at least one dimension belongs to $\{1, 8, 32\}$. Because combinations already present in the original 8×8 grid are not duplicated, this adds $(8 + 3)^2 - 8^2 = 57$ additional points.

Each entry in Table 1.3 maps one-to-one to a standalone model file:

Models/CUDA/Linear/Linear_<input_size>_<output_size>.pt

(for example, `Linear_128_2048.pt`), executed at batch size 1. In total, the Linear suite comprises $64 + 57 = 121$ distinct benchmark models.

Generic Conv measurement matrix (Table 1.4). The convolutional space spans four dimensions: input channels, spatial feature-map resolution, output channels, and kernel/padding geometries. Table 1.4 visualizes the primary base configuration plane:

- **Rows represent the input channel depth:** $C_{\text{in}} \in \{1, 2, 4, 8, 16, 32, 64, 128, 256, 512\}$.

$C_{\text{in}} \setminus (H=W)$	32	64	128	256	512	1024	Pairs
1	•	•	•	•	•	•	6
2	•	•	•	•	•	•	6
4	•	•	•	•	•	•	6
8	•	•	•	•	•	•	6
16	•	•	•	•	•	—	5
32	•	•	•	•	•	—	5
64	•	•	•	•	—	—	4
128	•	•	•	•	—	—	4
256	•	•	•	—	—	—	3
512	•	•	—	—	—	—	2
Total	10	10	8	8	6	4	47

Table 1.4: Generic Conv measurement matrix. Rows denote input channel count (C_{in}) and columns denote square spatial image resolution ($H = W$). Each bullet (•) represents a valid base (C_{in} , resolution) pair and expands to 32 experiments (8 output channels $\times$ 4 kernel/padding settings). Dashes (—) denote combinations intentionally skipped due to memory and architecture bounds. Together with 80 small-spatial and 64 pointwise pruning experiments, the Conv suite totals 1648 experiments.

- **Columns represent the square spatial resolution:** $H = W \in \{32, 64, 128, 256, 512, 1024\}$.

A naive Cartesian product of 10 channel counts $\times$ 6 spatial resolutions would yield 60 pairs. However, modern computer-vision architectures do not combine deep channel stacks with massive spatial resolutions (such as 512 channels at 1024×1024), and allocating such intermediate activation tensors would trigger out-of-memory errors on edge devices. Therefore, the launcher enforces resolution caps: channels 1–8 permit resolutions up to 1024; channels 16–32 up to 512; channels 64–128 up to 256; channel 256 up to 128; and channel 512 up to 64. Combinations exceeding these caps are skipped (marked by dashes ‘—’ in Table 1.4), leaving exactly 47 valid base pairs (marked by bullets ‘•’).

Crucially, unlike the Linear matrix where each bullet represents a single experiment, **each bullet (•) in Table 1.4 expands multiplicatively into 32 distinct benchmarking experiments**. Each valid (C_{in} , resolution) pair is crossed with:

1. **8 output-channel values:** $C_{\text{out}} \in \{1, 8, 16, 32, 64, 128, 256, 512\}$.
2. **4 kernel and padding configurations:** $(k = 3, p = 0)$, $(k = 3, p = 1)$, $(k = 5, p = 0)$, and $(k = 5, p = 1)$ (with stride fixed to 1).

This 32-fold expansion produces $47 \times 8 \times 4 = 1504$ generic convolutional experiments. Each model follows the naming scheme:

`Conv_<input_channels>_<image_size>_<kernel>_<padding>_<out_channels>.pt`

(for example, `Conv_256_128_3_1_512.pt` or `Conv_512_64_3_1_512.pt`).

Finally, two specialized pruning extensions supplement this generic matrix:

- **Small-spatial 3×3 grid** (80 experiments): tests spatial size 2×2 with padding 1 across all 10 input channels and 8 output channels ($10 \times 8 = 80$).
- **Pointwise 1×1 grid** (64 experiments): tests 1×1 kernels ($p = 0$) across channels $\{1, 8, 64, 512\}$ and spatial sizes $\{2, 8, 32, 64\}$ ($4 \times 4 \times 4 = 64$).

Together, the Conv suite schedules $1504 + 80 + 64 = 1648$ total experiments.

For a targeted experiment combining a high channel count with the largest spatial size still allowed for it, a good candidate is `Conv_256_128_3_1_512.pt`: 256 input channels, a 128×128 feature map, a 3×3 kernel, padding 1, and 512 output channels. If the goal is the maximum channel count instead, `Conv_512_64_3_1_512.pt` is the highest-channel generic point in the default campaign; its spatial size is capped at 64 by design.

Note that `all` covers the basic layer suites and LeNet; the ResNet suites are added explicitly, e.g. `-suite all,resnet18,resnet50`. On a Jetson-class board the full list above is a multi-day unattended run — this is why the skip-on-COMPLETE behavior matters so much: the campaign can be interrupted, the board serviced, and the run resumed exactly where it stopped, even across days.

During this work we ran the Linear, Conv, attention and rotary suites plus the LeNet and ResNet-18 decompositions on the Jetson Nano, the Raspberry Pi 5 and the AGX Orin; the ResNet-50 suite and the batch-1 ResNet-18 points on the Nano remain open (the Nano broke during the campaign, see Section 1.6).

1.4.5 Manual fallback

If SSH is not an option, or you prefer recording with the TI graphical tool, you can still run the board side alone. `run_manager.py` does the warm-up, calibration and bursts, then pauses and waits for you to start and stop the recording by hand (`-wait_for_acquisition`):

```
python run_manager.py --backend cuda \
  --model Models/CUDA/Linear/Linear_64_64.pt \
  --batch-size 1 --wait_for_acquisition
```

The model filename encodes the shape: `Linear_64_64.pt` is a linear layer with 64 inputs and 64 outputs.

1.5 Processing the results

Raw CSV files are not meant to be read by hand. To turn them into per-measurement visualization plots and a consolidated summary CSV table with average power and energy for each model, run:

```
MPLBACKEND=Agg python processing_report/process_and_visualize.py \
  --data-dir measurements/runs/BOARD_LABEL \
  --plot-dir measurements/Plot/BOARD_LABEL \
  --summary-dir measurements/lookup_summaries/summaries \
  --output-name BOARD_LABEL.csv
```

By default, the script reads all colocated CSV and JSON manifest pairs in `-data-dir`, generates per-measurement plot PDFs in `-plot-dir` (defaulting to `measurements/Plot/<data_dir_name>`), and writes the consolidated summary table directly into `measurements/lookup_summaries/summaries/<BOARD_LABEL>.csv`. The summary destination is controlled by `-summary-dir` (defaulting to `measurements/lookup_summaries/summaries`) and `-output-name` (aliased as `-summary-name`, defaulting to `<data_dir_name>.csv`).

Only finished experiments (`status: COMPLETE` in the manifest) are processed; incomplete ones are listed and skipped. For each burst, the script takes the samples inside the active region, subtracts the board's idle consumption (measured during the quiet window at the end of the recording), and integrates what is left over time. The result is the net energy of the layer itself, which is exactly the number you care about.

Before any statistic is computed, the samples inside each active region are *trimmed*. Raw bursts always contain a few samples that do not represent steady state: the first ones still carry the ramp-up of the workload, and the extreme power values are usually transient spikes or dips. For our campaigns we therefore discarded:

- the **first 10% of samples in time** inside each active region (`-discard-initial-samples` with `-initial-trim-percentage 10`), so the workload ramp-up never enters the average;
- the **lowest 10% and highest 10% of samples in power** (`-tail-trim-percentage 10`), cutting both tails of the power distribution and keeping only the steady-state plateau.

Trimming is applied per cycle, and only when the cycle ran long enough to afford it — the processor skips it entirely below a minimum number of inferences (`-min-inferences-for-trimming`, 10 by default), where every sample is precious. The code’s out-of-the-box defaults are more conservative (1% tails, no initial discard unless the flag is passed), so to reproduce our processing exactly, say it on the command line:

```
MPLBACKEND=Agg python processing_report/process_and_visualize.py \
--data-dir measurements/runs/BOARD_LABEL \
--plot-dir measurements/Plot/BOARD_LABEL \
--summary-dir measurements/lookup_summaries/summaries \
--output-name BOARD_LABEL.csv \
--discard-initial-samples --initial-trim-percentage 10 \
--tail-trim-percentage 10
```

If a burst looks noisy even after trimming, `-filter-after-discard` applies a Hampel filter (spike replacement based on a robust local standard deviation) followed by a short rolling mean — use it sparingly, and remember that every cleaning choice is visible in the per-measurement PDFs the script produces, so always give them a look before trusting the summary.

The last step produces the lookup table used by JouleGrad (chapter 2), starting from one or more processed summaries:

```
python -m processing_report.build_energy_lookup_table \
measurements/lookup_summaries/summaries/BOARD_LABEL.csv \
--output measurements/lookup_summaries/BOARD_LABEL_energy_lookup.csv
```

1.6 [Future work] Does the energy of a network equal the sum of its parts?

This section is, in a very literal sense, **the work to be done in the future**: what follows is not a finished result but the most important open question this project leaves behind, stated with all the evidence collected so far.

All the measurements above target single operations in isolation. The hope behind the whole pipeline is that the energy of a full network is simply the sum of the energies of its layers — if that were true, JouleGrad’s lookup-table estimates would compose perfectly. We checked this directly with `processing_report/summarize_architecture.py` (run it with the `banera_pt` conda environment from the repository root, no arguments needed): for each board, it compares the measured energy of the complete model against the sum of its standalone operations, weighted by how many times each operation is called.

The answer is: *almost, but not exactly* — and how much “not exactly” depends on the board:

- **LeNet on Raspberry Pi 5** (batch 1): the complete model costs 1.016 mJ per sample, but its 11 standalone operations sum to only 0.771 mJ. The sum covers just 75.8% of the composed model — about a quarter of the energy is unaccounted for.
- **LeNet on Jetson Nano** (batch 1): complete 1.023 mJ versus 0.959 mJ summed — the sum covers 93.8%, a much smaller gap.

- **ResNet18 on Jetson Nano** (batch 8): complete 49.6 mJ versus a comparable sum of 48.8 mJ — coverage 98.3%. One caveat: batch 1 is what actually matters, because the future use cases (JouleGrad and JouleNAS) always estimate single-sample inference. We could not test ResNet18 on the Nano at batch 1 because the board broke during the campaign — repeating this comparison at batch 1 on a replacement board is part of the future work.
- **ResNet18 on AGX Orin** (batch 1): the sign flips. The complete model (24.2 mJ) is *cheaper* than the comparable standalone sum (25.3 mJ) — coverage 104.5%.
- **Late-stage finding — ResNet-18 on Raspberry Pi 5 (CIFAR-10, batch 1):** During the final cross-layer validation with JouleNAS, an interesting shift was discovered on CIFAR-10 (32×32). Because all intermediate feature maps comfortably fit within the 2 MB L3 cache of the Cortex-A76 cores, memory traffic to DRAM was eliminated; here the complete model consumed 28.38 mJ per inference, approximately 19.3% *less* than the standalone table sum (35.15 mJ).
- **Late-stage finding — ResNet-18 on AGX Orin (CIFAR-10, batch 32):** In the subsequent cross-batch validation campaign on the NVIDIA Jetson AGX Orin, we investigated whether increasing batch size to 32 would amortize kernel launch latency and eliminate the composition discrepancy. The empirical result was striking: the complete dense ResNet-18 consumed 1.368 mJ per sample (total batch energy 43.78 mJ), matching the standalone lookup table sum (1.372 mJ) within an astonishing 0.3% margin (coverage: 99.7%). Rather than a definitive proof, this provides an encouraging hint to consider: as batch size increases, framework and kernel launch overheads become amortized, suggesting that batching may improve composability between isolated layer profiles and full-network energy.

Our working **hypothesis** is that the gap is a **cache effect**. Measured standalone, each operation runs in a loop over its own tensors: weights and activations stay hot in the cache hierarchy, and the measured energy reflects mostly computation. Inside the full network, each layer receives its input from the previous one, and the working set of the whole model is larger than what the caches can hold — data keep moving between cache levels and main memory, and that movement costs energy that standalone measurements never see. The Raspberry Pi 5, with the smallest caches of the three boards, shows exactly the largest gap; the AGX Orin, with a much larger cache hierarchy, can instead exploit producer–consumer locality between consecutive layers and the composition turns favorable. This is a hypothesis supported by the sign and magnitude of the observed gaps — it has not been verified directly (e.g. with hardware performance counters), and verifying it is part of the future work listed below.

Keep the gap in mind when reading JouleGrad’s estimates: they are built from standalone measurements, so on cache-poor boards they will underestimate the energy of a large composed network by an amount that, as the Pi 5 case shows, can reach 20–30%.

1.6.1 Open directions

This gap is the main open problem of the measurement layer. If you pick up the project from here, this is where your work matters most:

- **Confirm the cache explanation.** Repeat the standalone versus complete comparison while collecting cache-miss counters (`perf` is available on all our Linux boards) and check whether the gap correlates with last-level-cache misses. Batch size is a natural knob: it changes the ratio between compute and data movement, and the LeNet multi-batch rows in the script’s output already show the gap shrinking as the batch grows.

- **Quantify the gap across boards and models.** Extend the comparison to the remaining boards and to other architectures, so the correction becomes a function of hardware and model size rather than a handful of anecdotal numbers.
- **Feed the correction back into the estimator.** If the gap turns out to be systematic, JouleGrad’s lookup-based estimates should include a composition term — even a simple per-board multiplicative factor would already fix most of the error — and JouleNAS searches would then rank architectures by their true composed energy instead of the sum of parts.

None of this blocks the current pipeline: on the two Jetson boards the gaps measured so far are within about 5%. But the Pi 5 case shows the error can grow quickly on cache-poor hardware, and closing it is what turns a good per-layer estimator into a trustworthy whole-network one.

1.7 Tests and troubleshooting

The `tests/` directory contains automated checks (`python -m pytest tests/`). One honest warning: those tests were generated with an LLM and have *not* been fully verified against the current code, so treat a failing test as “worth investigating”, not as proof that the measurement pipeline is broken.

When something goes wrong, check the most common causes first:

1. **Nothing is measured (flat line in the trace):** the board is not powered through the INA226 shunt, or the PC is looking at the wrong serial device. Re-check the wiring and pass `-port` explicitly.
2. **The run fails immediately, before any measurement:** it is almost always SSH. Run the `SSH_OK` preflight from the configuration section — remember, the automation cannot type passwords.
3. **Exit code 2:** the model ran but the recording is invalid. Open the manifest and look at `quality_status` and the clock-uncertainty fields; usually re-running with the same label is enough.
4. **Bursts look too short in the trace:** this should not happen with the default settings (validation exists exactly for this). If you changed them, raise `-target_burst_seconds`.
5. **The board gets hot and the numbers drift:** give the board more rest — increase `-sleep_time` or use `-experiment-cooldown-seconds`. Keep in mind that energy measurement is, to some extent, also temperature measurement.

For the full protocol details and the complete CLI parameter reference (defaults, units, and whether each parameter ends up in the measured energy), see the long-form notes in the repository:

- `docs_new/adaptive_burst_measurement.md` — protocol and the parameter reference table (Section 9);
- `docs_new/experiment_campaign.md` — the campaign matrix and launcher examples.

Chapter 2

JouleGrad: Differentiable energy-estimation layer

JouleGrad is a small Python library that turns JouleQuest measurements into *differentiable* energy estimates. It reads an `energy_lookup_table.csv` and answers queries like "how many millijoules does a Linear 128 $\rightarrow$ 96 layer cost?" as PyTorch tensors. This means the answer has a gradient, so you can use it directly as a regularization term during training. It is strictly an API, with no command-line interface or measurement logic. The project is publicly hosted on GitHub at <https://github.com/danielefam/joulegrad>.

You can find the official documentation in the `docs/` directory of the repository. Each file covers a specific topic:

- `API.md` — User reference for the Python API, including classes, methods, tensor shapes, and examples.
- `LOOKUP_SCHEMA.md` — The schema of the `energy_lookup_table.csv`, covering columns, units, and measurement details.
- `INTERPOLATION.md` — Explains the multilinear interpolation logic used over the lookup grid.
- `OPTIMIZATION.md` — Details the improvements over the old `energy_estimator` repository (see Section 2.1).
- `OUT_OF_RANGE_POLICIES.md` — Explains how the library handles unsupported coordinates (errors, warnings, clamping, or extrapolation).
- `MIGRATION_FROM_ENERGY_ESTIMATOR.md` — A guide for migrating from the older `energy_estimator` package.

This chapter introduces the core concepts. For function signatures and practical usage details, check the documentation.

2.1 Improvements over `energy_estimator`

JouleGrad replaces an older package (`energy_estimator`) that used bundled Excel tables. The rewrite fixes three main issues that surfaced when using real JouleQuest measurements:

- **No more silent extrapolation.** (See Section 2.2) The old package silently extrapolated beyond the measured range without warning. JouleGrad makes this explicit: the default `out_of_range="error"` rejects unsupported coordinates. You can still choose approximations like `warn`, `clamp`, or `extrapolate` if needed.

- **No synthetic zero anchor.** Old tables assumed a zero-width layer had zero energy cost. However, framework and launch overheads still exist. JouleGrad doesn't invent this data point; low-width behavior must be actually measured.
- **Explicit data sources.** Instead of selecting a bundled file by board name, you now provide the exact file path. This ensures there is no ambiguity about the data you are using.

For backward compatibility, the old class name is still available as an alias. The model regularizer, one-shot model estimate, and low-level interpolation functions remain public.

2.2 Handling out-of-range queries

A key detail for any estimator is what happens when you query a shape that was never measured. The old package made this choice for you quietly; JouleGrad lets you explicitly configure this via the `out_of_range` parameter in the constructor.

2.2.1 The problem with silent extrapolation

If a measured axis ended at $x = 4$ and you queried $x = 5$, the old `clamp_index` helper clamped the coordinate to pick the edge cell $[2, 4]$. However, it computed the interpolation fraction using the original $x = 5$:

$$t = \frac{x - x_{lo}}{x_{hi} - x_{lo}} = \frac{5 - 2}{4 - 2} = 1.5$$

A fraction outside $[0, 1]$ extends the edge cell as a straight line. Thus, `clamp` was actually doing **silent linear extrapolation** without any warnings. It was impossible to tell if a result was measured, interpolated, or extrapolated.

2.2.2 JouleGrad policies

JouleGrad clearly separates cell selection and interpolation weights, offering distinct policies via `EnergyEstimator(..., out_of_range=...)`:

- **error** (default): Raises a `ValueError` for unmeasured coordinates. Nothing is invented. This is the best choice for reproducible, published results.
- **extrapolate**: Replicates the old behavior of extending the edge trend. This is now an explicit choice and includes gradients. Note that the result might not stay positive or monotonic. If only one data point exists on an axis, making slope extrapolation impossible, it issues a warning and clamps to that point.
- **clamp**: Clips the coordinate to the measured range *before* computing interpolation weights. Out-of-range queries simply return the energy of the nearest edge point. The gradient is zero here, so an optimizer won't push the value back into bounds.
- **warn**: Similar to `clamp`, but prints a `RuntimeWarning` for each out-of-range query. Useful for debugging.

Sparse-grid handling: For missing internal points, `warn`, `clamp`, and `extrapolate` fall back to the nearest measured cell (or nearest Conv2d configuration). `warn` will log this, while the others remain silent.

If an axis only has one measured value, exact queries work in all modes. For unmeasured queries, JouleGrad clamps silently under `clamp`, warns and clamps under `warn` and `extrapolate`, and raises an error under `error`.

In `error` mode, a missing cell only causes a failure if it’s actually needed for interpolation. If you query an exact measured point, it succeeds even if nearby corners are missing.

Ideally, your measurements should cover all needed configurations so you can use `error` mode. In our experiments, we used `extrapolate` to keep training smooth while exploring sparse grid regions.

2.3 Performance architecture

During training, the energy estimator is called at every optimization step. Reading CSV files, looping over layers in Python, and moving data between the CPU and GPU would be too slow.

To maintain speed, JouleGrad follows a simple rule: **do the heavy work once during setup, and use fast PyTorch tensor operations for the training loop.**

2.3.1 Loading grids into PyTorch tensors

`EnergyEstimator` reads and validates the CSV data exactly once when initialized. The tabular data is converted directly into PyTorch tensors:

- A 2D grid for `Linear` layers (input $\times$ output channels).
- Dedicated grids for `Conv2d` configurations, grouped by kernel, stride, and padding.
- Grids for attention layers.

Once built, pandas is no longer needed, and all lookups use PyTorch. In non-strict modes (`warn`, `clamp`, `extrapolate`), missing grid cells are filled during startup, so the training loop never pauses to calculate approximations.

2.3.2 Automatic GPU caching

Lookup tables start in CPU memory. Moving them to the GPU on every batch would add overhead.

JouleGrad solves this automatically: the first time it sees GPU coordinates, it moves the lookup table to that device and caches it. Subsequent steps reuse this cached tensor with zero transfer overhead.

Because of this, **you should create the estimator object just once before training starts.**

2.3.3 Vectorized interpolation

Interpolating in D dimensions requires combining values at 2^D surrounding corners.

Instead of Python `for` loops, JouleGrad vectorizes this calculation:

- Query coordinates are grouped into a single matrix.
- Bounding corners and interpolation weights are calculated together using tensor broadcasting.
- A single weighted sum computes the final energy.

Coordinate indices are detached from the computational graph, while fractional interpolation positions remain attached. This allows gradients to flow back to the active channel dimensions and pruning masks properly.

2.3.4 Batching layer evaluations

Calling the estimator individually for every layer in deep networks like ResNet-18 adds Python overhead.

To prevent this, JouleGrad provides batched methods: `linear_batch`, `conv2d_batch`, and `attention_batch`. The estimator:

1. Groups all `Linear` layers into a single query.
2. Groups `Conv2d` layers with matching kernel, stride, and padding.
3. Evaluates each group in one GPU operation and sums the results.

This batching drastically cuts down on overhead.

2.3.5 Preparing the model structure

To estimate the energy of a full model, JouleGrad needs its configuration. Rather than scanning the model tree on every batch:

- JouleGrad traverses the model’s modules once during initialization and saves a list of relevant layers and shapes.
- During training, providing new channel masks instantly calculates the energy without re-scanning.

For standard feed-forward networks, this is automatic. For models with residual connections (like ResNet in JouleNAS), where multiple layers share the same pruning mask, the topology is explicitly defined so shared masks are correctly handled.

2.3.6 Fast loss calculation vs. detailed diagnostics

The training loop only needs a single scalar: total energy. Tracking per-layer breakdowns at every step is wasteful. JouleGrad separates these tasks:

```
# 1. Fast path for training loss: returns only the total energy
energy_mj = prepared(masks)

# 2. Diagnostic path: returns a dictionary with per-layer details
details = prepared.estimate(masks)
```

Use the fast path for training and the diagnostic path for logging and evaluation.

2.3.7 Floating-point precision

By default, lookup tables use double precision (`torch.float64`) for high accuracy. If you prefer the speed of single precision, you can easily convert the tables to `float32`. Coordinate conversions stay within the PyTorch autograd graph in both cases.

2.4 Usage

JouleGrad requires Python ≥ 3.10 , PyTorch, and pandas. It can be installed directly from GitHub:

```
python -m pip install git+https://github.com/danielefam/joulegrad.git
```

Alternatively, when developing locally on a clone of the repository:

```
python -m pip install -e .
```

2.4.1 Layer-level queries

The `linear`, `conv2d`, and `attention` methods return scalar tensors in mJ per inference. They are differentiable in every coordinate. Vectorized `*_batch` versions are also available:

```
import torch
from joulegrad import EnergyEstimator

estimator = EnergyEstimator("energy_lookup_table.csv",
                             out_of_range="error")

active_outputs = torch.tensor(96.0, requires_grad=True)
energy_mj = estimator.linear(128, active_outputs)
energy_mj.backward()
print(float(energy_mj), active_outputs.grad)
```

2.4.2 Model-level queries

`prepare_model` scans the selected modules once. Afterward, passing channel masks to the prepared object returns the total estimated energy, with gradients flowing back to the masks:

```
prepared = estimator.prepare_model(
    model,
    input_shapes={"features.0": (1, 3, 32, 32)},
    module_names=("features.0", "classifier"),
)

energy_mj = prepared(masks)          # differentiable total
details = prepared.estimate(masks)  # per-layer breakdown
```

For a quick, one-off check (like in a notebook), you can use `estimator.estimate_model(...)`. This helper prepares the model and runs the estimation in one step. For training, always use the two-step `prepare_model` approach so you don't scan the architecture on every batch.

In architectures like ResNet, skip connections sum the outputs of different branches. For this to work, both branches must share the exact same number of channels and pruning mask. PyTorch modules do not expose this topology automatically. Therefore, JouleNAS explicitly tracks these shared masks across residual blocks and calls JouleGrad's batch functions directly.

For more advanced usage, consult the full API reference and guides in the `docs/` directory.

Chapter 3

JouleNAS: Environmentally-aware optimization layer

JouleNAS takes a trained model, freezes its weights, and learns *which channels to keep* by optimizing channel masks. While this masking formulation was originally proposed to find unbiased subnetworks [1], here we repurpose it to discover energy-efficient architectures. The loss function includes the energy estimate from JouleGrad:

$$L = L_{\text{CE}} + \lambda_E \hat{E},$$

where:

- $L_{\text{CE}} \geq 0$ is the standard classification loss;
- $\hat{E}$ is the estimated inference energy in millijoules (mJ);
- $\lambda_E \geq 0$ (**-energy-weight**) controls how much we penalize energy use. If $\lambda_E = 0$, we only optimize for accuracy (the baseline).

Depending on the **-energy-mode** flag, $\hat{E}$ uses either the discrete count of active channels or the continuous sum of channel probabilities (see section 3.2.1).

The repository includes three main setups: **SimpleConv** on synthetic bars or MNIST (for learning and debugging), **ResNet-18 on CIFAR-10** (using a pretrained checkpoint), and **ResNet-18 on Imagenette** (trained from scratch first, then optimized).

The repository includes several documentation files to guide you:

- **README.md** — The project’s entry point, covering installation and the commands to run experiments.
- **TUTORIAL.md** — A step-by-step guide from a pretrained model to the final pruned network, showing how the training loop and masks work together.
- **docs/RESNET18_ICPR_ENERGY_METHODODOLOGY.md** — Explains the methodology behind the ResNet-18 search, how we penalize energy, and how we optimize for actual hardware costs.
- **docs/RESNET18_CIFAR10_ICPR_COMPARISON.md** — Compares a standard baseline with an energy-aware run on CIFAR-10.

While this chapter explains the core ideas and how to run them, you can check those documents for exact training details and deeper insights into the energy search. Note that, unlike JouleGrad which is publicly hosted on GitHub, the JouleNAS repository is and will remain private at the moment. Its formulation, hyperparameter controls, and experimental results are nonetheless documented in detail in this report to serve as an exact reference.

3.1 How the mask optimization works

During this phase, only the mask logits are updated; the main network weights and BatchNorm statistics remain frozen. For each channel i , the gating mechanism relies on five variables:

- $m_i \in (-\infty, +\infty)$ (**mask logit**): A learnable score for channel i , updated via gradient descent. A positive score keeps the channel open, while a negative score drops it.
- $\tau \in (0, +\infty)$ (**temperature**): A scaling parameter that makes the sigmoid curve steeper over time. It starts at 1.0 and halves every 10 epochs ($1.0 \rightarrow 0.5 \rightarrow 0.25 \dots$).
- $p_i \in (0, 1)$ (**channel probability**): The soft probability of keeping channel i , computed with a standard sigmoid:

$$p_i = \sigma(m_i/\tau) = \frac{1}{1 + e^{-m_i/\tau}}.$$

When $m_i = 0$, $p_i = 0.50$.

- $\theta \in [0, 1]$ (**threshold**): The cutoff that turns probabilities into hard decisions. We start at 0.49 for the first 20 epochs, then tighten it to 0.50.
- $g_i \in \{0, 1\}$ (**binary gate**): The actual switch for channel i . We compute it with the step function H :

$$g_i = H(p_i - \theta) = \begin{cases} 1 & \text{if } p_i \geq \theta \\ 0 & \text{if } p_i < \theta \end{cases}$$

During the forward pass, channel i 's feature map is multiplied by its gate g_i . If $g_i = 1$, the channel passes normally; if $g_i = 0$, it is completely zeroed out. In the backward pass, gradients flow through the step function back to the logits m_i using the straight-through estimator (STE).

As training goes on and the temperature τ drops, the probabilities p_i polarize toward 0 or 1.

3.2 How the energy term is computed

The loss's second term, $\hat{E}$, is the JouleGrad estimate for the masked model's inference energy. Let's look at how we define effective layer sizes and why this estimate is differentiable.

3.2.1 Discrete versus continuous energy modes

JouleNAS offers two ways to compute effective layer sizes from the learned masks:

1. **Discrete mode** (-energy-mode discrete, default): The effective output width $d_{\text{eff}}^{(\ell)}$ is the exact integer count of active channels:

$$d_{\text{eff}}^{(\ell)} = \sum_{i=1}^{C_\ell} g_i^{(\ell)} = \sum_{i=1}^{C_\ell} H(p_i^{(\ell)} - \theta),$$

where C_ℓ is the total number of channels in layer ℓ . This looks up the energy of the pruned model using only the open channels. During the backward pass, gradients flow through the straight-through estimator: $\partial g_i / \partial m_i \approx p_i(1 - p_i)/\tau$.

2. **Continuous mode** (-energy-mode continuous): The effective output width $d_{\text{eff}}^{(\ell)}$ is the sum of the soft channel probabilities $p_i^{(\ell)}$:

$$d_{\text{eff}}^{(\ell)} = \sum_{i=1}^{C_\ell} p_i^{(\ell)} = \sum_{i=1}^{C_\ell} \sigma(m_i^{(\ell)}/\tau).$$

Here $d_{\text{eff}}^{(\ell)}$ can take any fractional value between 0 and C_ℓ , representing the expected number of open channels. Since JouleGrad uses multilinear interpolation, it gracefully handles fractional dimensions to return a smooth energy estimate $\hat{E}$. In this branch, the non-differentiable step function H is completely bypassed, avoiding abrupt step jumps and yielding a clean, exact analytical gradient for the energy penalty: $\partial \hat{E} / \partial m_i = (\partial \hat{E} / \partial d_{\text{eff}}) \cdot (p_i(1 - p_i) / \tau)$. (Note that while the network’s classification forward pass still uses binary gates g_i to evaluate real pruned accuracy via STE, the energy branch itself requires no surrogate approximation).

When a layer’s input comes from another masked layer, its effective input width $d_{\text{eff},\text{in}}^{(\ell)}$ is determined by that preceding mask. Other dimensions like spatial size, strides, and kernel size remain fixed by the architecture.

3.2.2 From effective sizes to energy: multilinear interpolation

Each layer’s energy is looked up in our measured table. Because the effective width is rarely an exact point on the measured grid, JouleGrad interpolates the values. For a two-axis layer like a Linear layer (with effective input x and output y), the estimated energy $\hat{E}_\ell$ is a bilinear combination of the four nearest measured corners $E_{a,b}$:

$$\hat{E}_\ell = \sum_{a \in \{\text{lo}, \text{hi}\}} \sum_{b \in \{\text{lo}, \text{hi}\}} w_a w'_b E_{a,b},$$

where $E_{a,b}$ are the measured energies in mJ, and the weights w_a, w'_b are simple distance-based fractions:

$$w_{\text{hi}} = \frac{x - x_{\text{lo}}}{x_{\text{hi}} - x_{\text{lo}}}, \quad w_{\text{lo}} = 1 - w_{\text{hi}}, \quad w'_{\text{hi}} = \frac{y - y_{\text{lo}}}{y_{\text{hi}} - y_{\text{lo}}}, \quad w'_{\text{lo}} = 1 - w'_{\text{hi}}.$$

This extends to three axes for Conv2d layers (input channels, output channels, and image size). Summing $\hat{E}_\ell$ over all layers gives the total model energy:

$$\hat{E} = \sum_{\ell} \hat{E}_\ell(d_{\text{eff},\text{in}}^{(\ell)}, d_{\text{eff},\text{out}}^{(\ell)}, \dots).$$

3.2.3 Why gradients flow to the mask logits

Because the total loss decomposes into two terms:

$$L = L_{\text{CE}} + \lambda_E \hat{E},$$

the gradient with respect to each mask logit m_i splits into two distinct computational branches in PyTorch’s computational graph:

$$\frac{\partial L}{\partial m_i} = \frac{\partial L_{\text{CE}}}{\partial m_i} + \lambda_E \frac{\partial \hat{E}}{\partial m_i}.$$

- **The task loss branch** ($\partial L_{\text{CE}} / \partial m_i$): During the forward pass through the neural network, feature maps are multiplied by the binary gate $g_i = H(p_i - \theta) \in \{0, 1\}$ in both modes. This ensures that classification accuracy reflects true, hard-pruned sub-networks without soft leakage. Because the step function H has a zero derivative almost everywhere, back-propagating $\partial L_{\text{CE}} / \partial m_i$ always relies on the Straight-Through Estimator (STE), replacing $\partial g_i / \partial m_i$ with $\partial p_i / \partial m_i = p_i(1 - p_i) / \tau$.
- **The energy penalty branch** ($\partial \hat{E} / \partial m_i$): This branch evaluates the hardware energy of the candidate architecture using JouleGrad. Here the two modes diverge:

- In **discrete mode**, the energy table is queried with the integer count of open gates $d_{\text{eff}} = \sum g_i$. The energy gradient $\partial \hat{E} / \partial m_i$ must therefore also pass through the binary step function g_i , requiring the STE approximation for the energy term as well.
- In **continuous mode**, the energy table is queried with the sum of soft probabilities $d_{\text{eff}} = \sum p_i$. The binary step function is completely bypassed in this branch. Because $p_i = \sigma(m_i / \tau)$ is smooth and JouleGrad’s multilinear interpolation is continuously differentiable almost everywhere, the energy gradient $\partial \hat{E} / \partial m_i$ is **exact and analytical** via the standard chain rule:

$$\frac{\partial \hat{E}}{\partial m_i} = \frac{\partial \hat{E}}{\partial d_{\text{eff}}} \cdot \frac{\partial d_{\text{eff}}}{\partial p_i} \cdot \frac{\partial p_i}{\partial m_i} = \frac{\partial \hat{E}}{\partial d_{\text{eff}}} \cdot 1 \cdot \frac{p_i(1 - p_i)}{\tau}.$$

- **Differentiable interpolation weights:** The multilinear interpolation weights w are simple linear fractions of d_{eff} , making $\partial \hat{E}_\ell / \partial d_{\text{eff}, \text{in}}$ and $\partial \hat{E}_\ell / \partial d_{\text{eff}, \text{out}}$ readily available in closed form.

Together, these two paths give the optimizer a clear trade-off: the classification loss works to keep channels that protect accuracy, while the energy penalty pushes to drop channels that cost too much power. In continuous mode, this energy push is completely smooth and exact, which prevents sudden jumps and keeps training stable.

3.3 Running an experiment

For authorized users with repository access, install the local JouleNAS package alongside public JouleGrad (available on GitHub at <https://github.com/danielefam/joulegrad>):

```
python -m pip install git+https://github.com/danielefam/joulegrad.git
python -m pip install -e '[train]'
```

Alternatively, JouleNAS declares JouleGrad as a direct Git dependency in `pyproject.toml`, so installing JouleNAS directly will automatically fetch and install JouleGrad from GitHub:

```
python -m pip install -e '[train]'
```

Then run the CIFAR-10 search:

```
joule-nas train-resnet18 \
--pretrained-checkpoint /path/to/cifar10_resnet18.pth \
--energy-lookup-table /path/to/energy_lookup_table.csv \
--energy-weight 0.005 \
--energy-mode continuous \
--logit-init 2.0 \
--energy-warmup-epochs 5 \
--metrics-csv runs/metrics.csv \
--output-checkpoint runs/masks.pth
```

3.3.1 Optimization controls and hyperparameters

You can tweak the mask search with a few key CLI options:

- **-energy-mode** (**discrete** or **continuous**): Sets how layer widths are calculated for the energy estimate $\hat{E}$. Discrete mode uses integer counts of active channels ($d_{\text{eff}} = \sum g_i$), while continuous mode uses soft probabilities ($d_{\text{eff}} = \sum p_i$). (Note: the forward pass always uses hard binary gates, regardless of this mode).

- **-logit-init** (default 0.0): Sets the starting value m_i for all mask logits. Starting at 0.0 means channel probabilities begin at $p_i = 0.50$. Higher values (like 2.0) start the channels mostly open ($p_i \approx 0.88$).
- **-energy-warmup-epochs** (default 0): Slowly ramps up the energy penalty from 0 to λ_E :

$$\lambda_E(t) = \lambda_E \cdot \min \left(1.0, \frac{t}{\text{warmup_epochs}} \right).$$

This gives the network time to find the important channels before the energy penalty kicks in fully.

For Imagenette, the process is split into two SLURM scripts in `slurm/`:

1. **pretrain_resnet18_imagenette.sh**: Trains a dense ResNet-18 from scratch for 80 epochs and saves the best checkpoint.
2. **train_resnet18_imagenette.sh**: Loads that checkpoint and optimizes the pruning masks for your chosen energy penalty λ_E .

This split means you only train the heavy baseline once, and can run many quick pruning sweeps using the same checkpoint.

A quick tip: your lookup table needs to cover the model’s operators at the chosen input size. You can pass **-energy-out-of-range error** to strictly enforce this, or use a softer JouleGrad policy like **extrapolate** if your table is incomplete (we used this one).

Each run outputs a metrics CSV (tracking accuracy, active channels, and energy) and a final mask checkpoint, which is ready for the deployment phase discussed in the Future Work chapter.

3.4 Experimental results and what they tell us

We evaluated JouleNAS by applying the energy-constrained mask optimization to ResNet-18 across two benchmark computer vision datasets: CIFAR-10 (32×32 image resolution) and Imagenette (224×224 image resolution). Experiments were conducted across two distinct edge hardware platforms with vastly different architectural and energy profiles:

- **NVIDIA Jetson AGX Orin**: an embedded GPU system with massive parallel compute capacity, consuming 18.51 mJ per dense CIFAR-10 inference and 19.32 mJ per dense Imagenette inference;
- **Raspberry Pi 5**: a commodity quad-core ARM Cortex-A76 edge CPU, consuming 35.15 mJ per dense CIFAR-10 inference and 56.41 mJ per dense Imagenette inference.

CIFAR-10 experiments start from a frozen pretrained checkpoint (92.30% baseline test accuracy), while Imagenette models are trained from scratch (88.46% baseline test accuracy) before mask optimization begins.

The following subsections examine the results in detail: first on the primary embedded GPU target (Section 3.4.1), followed by cross-hardware validation on the edge CPU (Section 3.4.2).

3.4.1 Embedded GPU target: NVIDIA Jetson AGX Orin

Table 3.1 summarizes the complete set of JouleNAS pruning runs on CIFAR-10 and Imagenette using the empirical lookup table measured on the NVIDIA Jetson AGX Orin across various penalty weights λ_E , optimization modes, warmup schedules, and logit initializations.

Table 3.1: Summary of JouleNAS pruning experiments on CIFAR-10 and Imagenette (evaluated on NVIDIA Jetson AGX Orin lookup).

Dataset	Configuration	λ_E	Mode	Dense Acc	Final Acc	Energy Frac	Active Frac
CIFAR-10	Control ($\lambda_E = 0$)	0.0	Disc.	92.30%	92.43%	81.73%	91.8%
CIFAR-10	Control ($\lambda_E = 0$)	0.0	Cont.	92.30%	92.43%	82.71%	92.1%
CIFAR-10	Standard	0.001	Cont.	92.30%	92.42%	66.09%	77.8%
CIFAR-10	Standard	0.002	Cont.	92.30%	92.36%	61.44%	70.3%
CIFAR-10	Warmup 10ep	0.002	Cont.	92.30%	92.15%	61.16%	71.1%
CIFAR-10	Standard	0.005	Disc.	92.30%	92.10%	58.27%	67.9%
CIFAR-10	Standard	0.005	Cont.	92.30%	91.80%	58.73%	66.4%
CIFAR-10	Standard	0.01	Disc.	92.30%	92.11%	57.20%	67.0%
CIFAR-10	Standard	0.01	Cont.	92.30%	91.94%	56.38%	63.8%
CIFAR-10	Standard	0.02	Disc.	92.30%	91.39%	41.20%	61.4%
CIFAR-10	Standard	0.02	Cont.	92.30%	91.98%	54.79%	62.0%
CIFAR-10	Aggressive	0.05	Cont.	92.30%	90.87%	43.52%	53.8%
CIFAR-10	Aggressive	0.05	Disc.	92.30%	10.00%	18.53%	8.2%
CIFAR-10	Warmup 10ep	0.05	Disc.	92.30%	10.00%	18.44%	8.3%
CIFAR-10	Aggressive	0.10	Disc.	92.30%	90.13%	47.43%	67.8%
CIFAR-10	Aggressive	0.10	Cont.	92.30%	10.00%	17.18%	11.5%
CIFAR-10	Warmup 10ep	0.10	Cont.	92.30%	89.65%	40.04%	50.9%
CIFAR-10	Init 2.0	0.10	Cont.	92.30%	92.30%	100.00%	100.0%
Imagenette	Control ($\lambda_E = 0$)	0.0	Disc.	88.46%	88.13%	96.95%	97.2%
Imagenette	Control ($\lambda_E = 0$)	0.0	Cont.	88.46%	88.25%	97.37%	97.5%
Imagenette	Low penalty	0.01	Disc.	88.46%	88.20%	77.00%	84.6%
Imagenette	Low penalty	0.01	Cont.	88.46%	87.46%	78.34%	85.9%
Imagenette	Moderate penalty	0.03	Disc.	88.46%	87.57%	66.47%	74.3%
Imagenette	Balanced	0.05	Disc.	88.46%	86.65%	59.05%	67.1%
Imagenette	Balanced	0.05	Cont.	88.46%	86.47%	57.85%	66.1%
Imagenette	Aggressive	0.10	Cont.	88.46%	84.66%	49.87%	59.5%
Imagenette	Aggressive	0.10	Disc.	88.46%	9.86%	17.72%	7.5%
Imagenette	Warmup 10ep	0.10	Disc.	88.46%	85.50%	51.56%	60.1%
Imagenette	Init 2.0	0.10	Disc.	88.46%	88.46%	100.00%	100.0%

The zero-penalty control: an implicit regularization

To set a clean baseline, we ran control experiments with no energy penalty ($\lambda_E = 0$). Here, the network only optimizes for accuracy ($L = L_{CE}$), ignoring energy entirely.

Figure 3.1 compares the zero-penalty ($\lambda_E = 0$) energy trajectories on CIFAR-10 and Imagenette across both target hardware platforms (NVIDIA Jetson AGX Orin and Raspberry Pi 5). The two datasets behave quite differently:

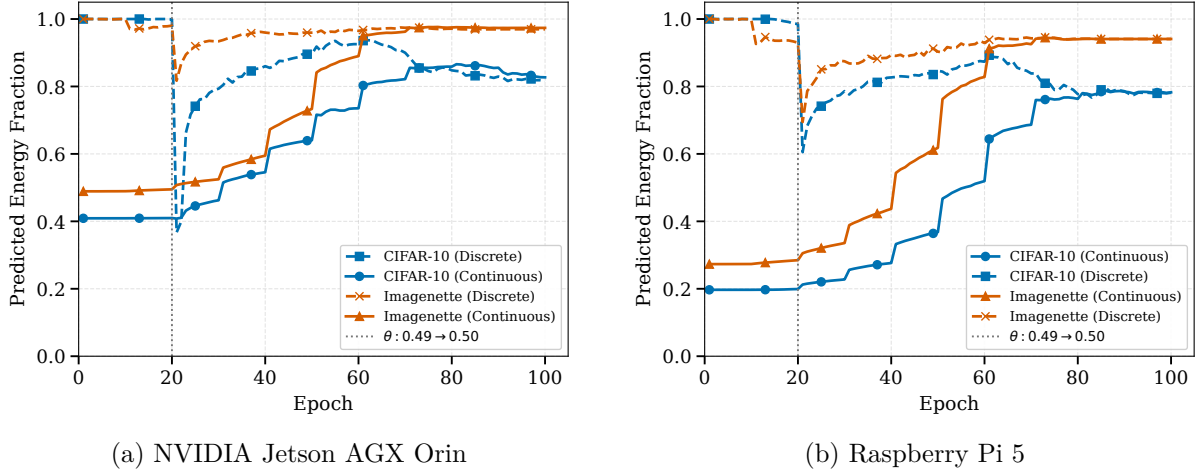


Figure 3.1: Control experiment ($\lambda_E = 0$) comparing CIFAR-10 and Imagenette predicted energy trajectories across epochs on NVIDIA Jetson AGX Orin (a) and Raspberry Pi 5 (b). On both platforms, Imagenette preserves nearly its entire energy footprint ($\approx 94\text{--}97\%$), while CIFAR-10 naturally sheds 18–22% of energy without any penalty.

- **Imagenette retains full capacity:** Across both devices and modes, the network keeps nearly all of its energy ($\approx 97\%$ on Jetson Orin, $\approx 94\%$ on Raspberry Pi 5). Imagenette’s high-resolution images require the full expressive power of ResNet-18, so almost every channel remains active to preserve accuracy.
- **CIFAR-10 naturally sheds energy:** Even without an explicit energy penalty, the network drops redundant channels, reducing predicted inference energy by over 18% on Jetson Orin and 22% on Raspberry Pi 5 while maintaining baseline test accuracy (92.43% vs. 92.30%).

CIFAR-10: energy and accuracy trade-off

Once we turn on the energy penalty ($\lambda_E > 0$), the network starts pruning useful channels to save power, giving us a smooth trade-off (see Table 3.1). Sweeping λ_E from 0.001 to 0.02 drops energy from 66% down to just 41% (a massive 59% saving), while keeping accuracy mostly intact above 91.3%.

Pushing λ_E to 0.10 exposes an interesting split: discrete mode holds steady (90.1% accuracy, 47.4% energy), while continuous mode collapses unless we use a warmup period (discussed below).

Figure 3.2 shows the training curves for the discrete run at $\lambda_E = 0.005$. Accuracy stays comfortably around 92% while the estimated energy drops smoothly over time.

Interestingly, Figure 3.2b shows that the energy fraction drops faster than the channel count (58.3% energy vs. 67.9% channels). This happens because early layers (with large 32×32 or 16×16 spatial sizes) cost much more energy per channel than later layers. The optimizer smartly targets the most expensive channels first, rather than treating them all equally.

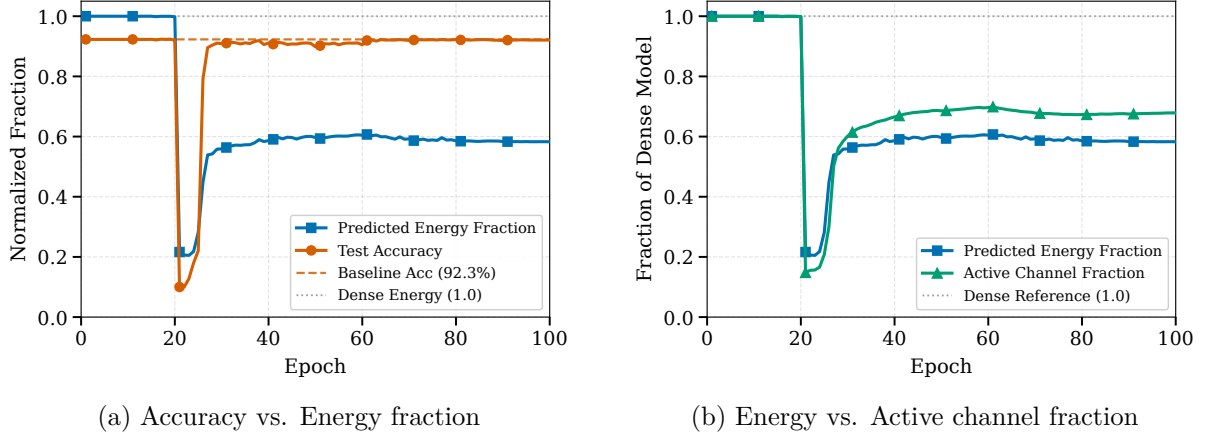


Figure 3.2: Training curves on CIFAR-10 ResNet-18 ($\lambda_E = 0.005$, discrete mode). Accuracy stays near the baseline while energy drops by roughly 41%.

Training instability and hyperparameter sensitivity

With moderate penalties ($\lambda_E \leq 0.05$ in continuous mode), the search is predictable and produces good trade-offs on both datasets. But if we push the penalty higher, the training dynamics become fragile and unpredictable (see Table 3.1 and Figure 3.3):

- **Opposite reactions at $\lambda_E = 0.10$:** On Imagenette, discrete mode collapses to random guessing (9.86% accuracy), while continuous mode stays robust at 84.66%. On CIFAR-10, it is the reverse: continuous mode collapses to 10.00%, while discrete mode recovers nicely to 90.13% accuracy.
- **Bizarre scaling:** On CIFAR-10 in discrete mode, the network collapses at a lower penalty ($\lambda_E = 0.05$), but succeeds at a harsher penalty ($\lambda_E = 0.10$).
- **Warmup is not always the solution:** A 10-epoch warmup saves discrete mode at $\lambda_E = 0.10$ on Imagenette, but fails to save discrete mode at $\lambda_E = 0.05$ on CIFAR-10.

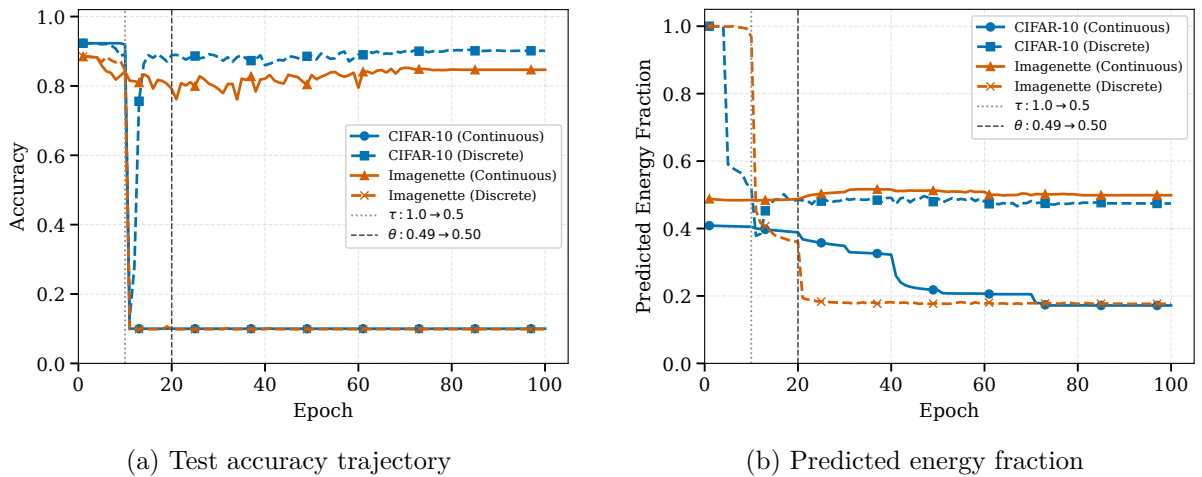


Figure 3.3: Aggressive pruning ($\lambda_E = 0.10$) dynamics across CIFAR-10 and Imagenette in continuous and discrete modes: (a) test accuracy and (b) predicted energy fraction. On Imagenette, discrete mode collapses while continuous remains stable; on CIFAR-10, continuous mode collapses while discrete recovers after epoch 11.

Hyperparameter sensitivity: logit initialization and warmup

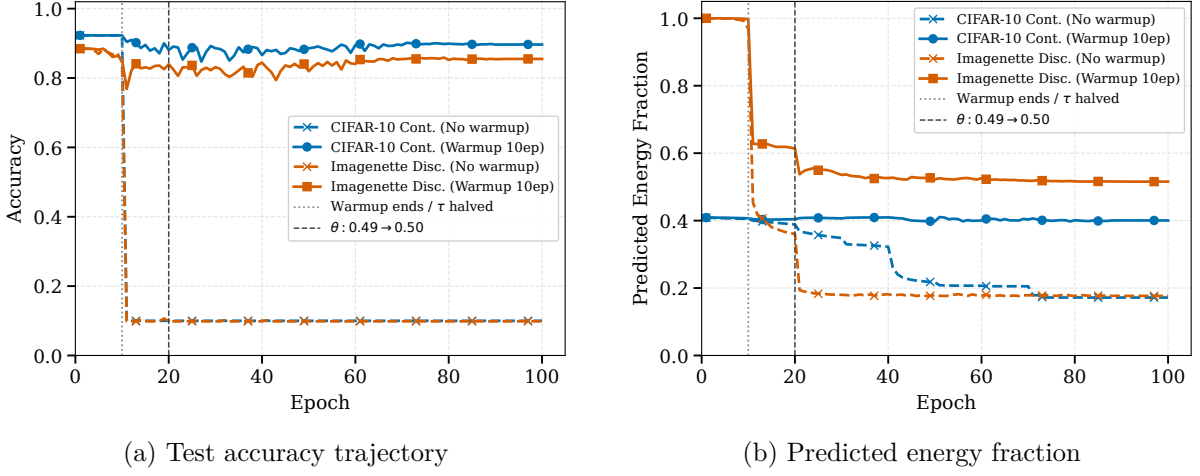


Figure 3.4: Effect of a 10-epoch linear energy penalty warmup on aggressive pruning ($\lambda_E = 0.10$): (a) test accuracy and (b) predicted energy fraction. Warmup successfully rescues CIFAR-10 continuous mode (recovering to 89.65%) and Imagenette discrete mode (recovering to 85.50%).

We also tested energy warmup and different logit initializations (Table 3.1):

- **Energy warmup:** At $\lambda_E = 0.10$ without warmup, both CIFAR-10 continuous mode and Imagenette discrete mode collapsed. Adding a 10-epoch warmup fixed these specific crashes (Figure 3.4), restoring accuracy to 89.65% and 85.50%, respectively. However, warmup did not prevent collapse for CIFAR-10 in discrete mode at $\lambda_E = 0.05$.
- **Logit initialization:** Starting logits at $m_i = 0$ means $p_i = 0.50$, right on the threshold of pruning. If we start them at $m_i = 2.0$ ($p_i \approx 0.88$) with $\lambda_E = 0.10$, the network keeps all channels open (100.0%), showing high sensitivity to the initial logit value.

The accuracy–energy Pareto frontier

Figure 3.5 combines all our successful runs into a Pareto frontier, mapping accuracy against energy consumption. This highlights three clear phases:

1. **The free pruning plateau:** Initially, we save energy for free. On CIFAR-10 ($\lambda_E = 0.005$), we cut energy by nearly 44% with zero drop in accuracy (92.26% vs. the 92.30% baseline). On Imagenette ($\lambda_E = 0.010$), we save 23% energy with a tiny drop (88.20% vs. 88.46%). This proves the model has a lot of redundant channels we don’t need.
2. **The gentle trade-off:** As we push λ_E up to 0.05, we get deeper savings for a small accuracy cost. For example, CIFAR-10 at $\lambda_E = 0.02$ uses only 41.2% of the original energy while keeping 91.39% accuracy (a $< 1\%$ drop). Imagenette at $\lambda_E = 0.05$ uses just 57.8% energy with 86.47% accuracy. These are great, practical sweet spots for real hardware.
3. **Rescued by warmup:** Without warmup, hitting $\lambda_E = 0.10$ causes crashes (e.g., continuous CIFAR-10 and discrete Imagenette drop to $\approx 10\%$). By adding a 10-epoch warmup, we rescue these runs and stretch the frontier even further, reaching 40.0% energy on CIFAR-10 (89.65% accuracy) and 51.5% energy on Imagenette (85.50% accuracy).

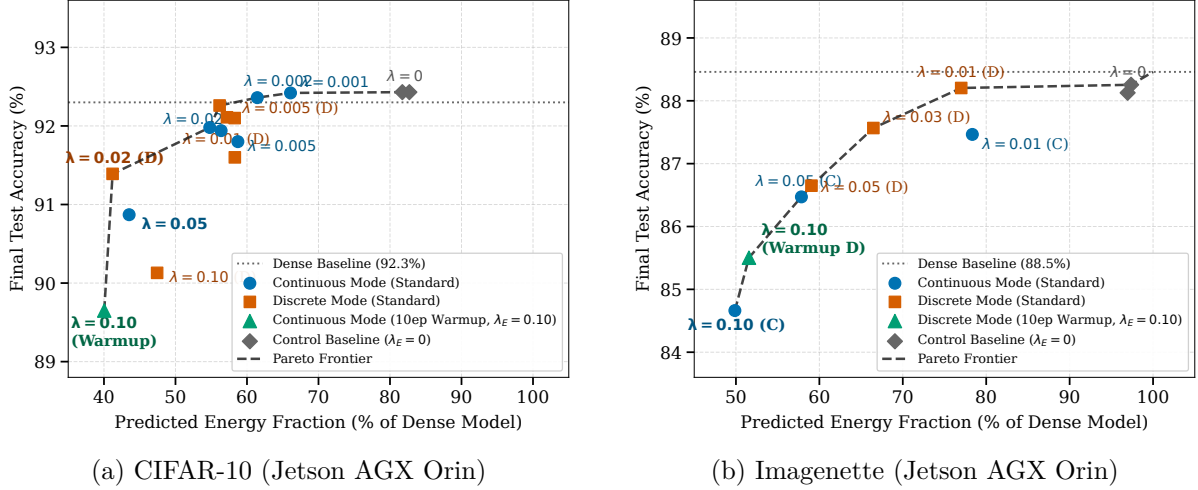


Figure 3.5: Empirical accuracy–energy Pareto frontier on the NVIDIA Jetson AGX Orin for (a) CIFAR-10 and (b) Imagenette. Dashed lines trace the optimal non-dominated trade-offs.

3.4.2 Cross-hardware generalization: evaluation on Raspberry Pi 5

To test whether JouleNAS generalizes beyond high-end embedded GPUs, we deployed the exact same pruning pipeline onto an edge CPU target: the Raspberry Pi 5 (quad-core ARM Cortex-A76 at 2.4 GHz), using the empirical lookup grid measured via JouleQuest.

Physical energy scaling and gradient pressure

Because the loss function couples task loss with absolute inference energy in millijoules:

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda_E \cdot \hat{E} \text{ [mJ]}, \quad (3.1)$$

the gradient magnitude $\nabla_m(\lambda_E \cdot \hat{E})$ scales directly with the physical energy consumption of the target hardware. On the Raspberry Pi 5, the dense ResNet-18 baseline consumes:

- **CIFAR-10:** 35.15 mJ per inference ($1.90\times$ higher than the 18.51 mJ on AGX Orin).
- **Imagenette:** 56.41 mJ per inference ($2.92\times$ higher than the 19.32 mJ on AGX Orin).

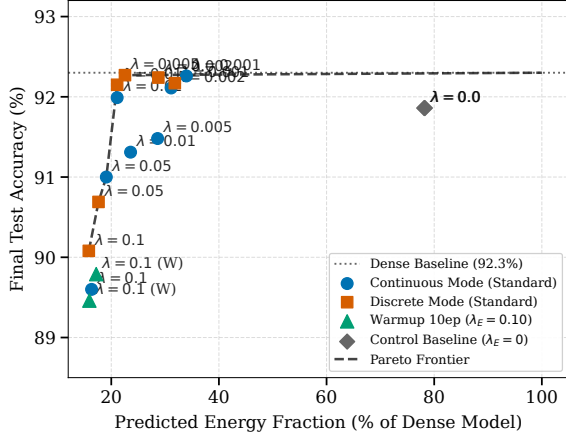
The accuracy–energy Pareto frontier on edge CPU

Table 3.2 provides the complete experimental summary across all penalty configurations on the Raspberry Pi 5, while Figure 3.6 maps the resulting accuracy–energy Pareto frontier.

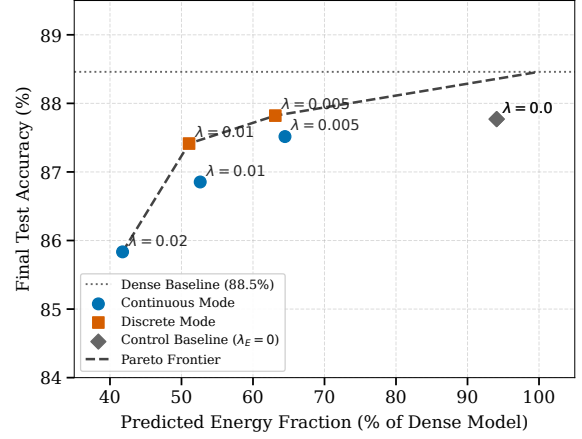
Cross-hardware architectural insights and stability regimes

Several key architectural and hardware-aware insights emerge from Table 3.2:

1. **Steeper Pareto gains at lower penalty on CIFAR-10:** Because each millijoule carries roughly twice the numerical weight on Pi 5 compared to AGX Orin, the network prunes much more aggressively at low values of λ_E . At $\lambda_E = 0.005$ in discrete mode, CIFAR-10 achieves an impressive 22.54% **energy fraction** (7.92 mJ) while maintaining a test accuracy of 92.27% (essentially identical to the 92.30% dense baseline). On AGX Orin, the same $\lambda_E = 0.005$ only reached 58.3% energy.



(a) CIFAR-10 (Raspberry Pi 5)



(b) Imagenette (Raspberry Pi 5)

Figure 3.6: Empirical accuracy–energy Pareto frontier on the Raspberry Pi 5 for (a) CIFAR-10 and (b) Imagenette. Dashed lines trace the empirical Pareto frontier across successful runs.

Table 3.2: Summary of JouleNAS pruning experiments on CIFAR-10 and Imagenette evaluated on the Raspberry Pi 5 lookup table (Dense CIFAR-10 energy: 35.15 mJ, Dense Imagenette energy: 56.41 mJ).

Dataset	Configuration	λ_E	Mode	Dense Acc	Final Acc	Energy Frac	Active Frac
CIFAR-10	Control ($\lambda_E = 0$)	0.0	Cont.	92.30%	91.86%	78.26%	93.0%
CIFAR-10	Control ($\lambda_E = 0$)	0.0	Disc.	92.30%	91.86%	78.15%	93.0%
CIFAR-10	Ultra-light	0.001	Cont.	92.30%	92.26%	33.96%	67.8%
CIFAR-10	Ultra-light	0.001	Disc.	92.30%	92.17%	31.81%	66.6%
CIFAR-10	Light	0.002	Cont.	92.30%	92.11%	31.12%	65.6%
CIFAR-10	Light	0.002	Disc.	92.30%	92.24%	28.75%	64.4%
CIFAR-10	Standard	0.005	Cont.	92.30%	91.48%	28.61%	63.3%
CIFAR-10	Standard	0.005	Disc.	92.30%	92.27%	22.54%	60.4%
CIFAR-10	Moderate	0.01	Cont.	92.30%	91.31%	23.59%	58.9%
CIFAR-10	Moderate	0.01	Disc.	92.30%	92.15%	21.09%	59.1%
CIFAR-10	Balanced	0.02	Cont.	92.30%	91.99%	21.09%	55.6%
CIFAR-10	Balanced	0.02	Disc.	92.30%	10.00%	2.51%	0.7%
CIFAR-10	High penalty	0.05	Cont.	92.30%	91.00%	19.08%	50.0%
CIFAR-10	High penalty	0.05	Disc.	92.30%	90.69%	17.63%	50.8%
CIFAR-10	Aggressive	0.10	Cont.	92.30%	89.60%	16.34%	45.1%
CIFAR-10	Aggressive	0.10	Disc.	92.30%	90.08%	15.83%	46.8%
CIFAR-10	Warmup 10ep	0.10	Cont.	92.30%	89.79%	17.20%	46.1%
CIFAR-10	Warmup 10ep	0.10	Disc.	92.30%	89.46%	15.93%	47.9%
CIFAR-10	Over-pruned	0.50	Disc.	92.30%	37.36%	4.94%	19.1%
Imagenette	Control ($\lambda_E = 0$)	0.0	Cont.	88.46%	87.77%	94.08%	97.1%
Imagenette	Control ($\lambda_E = 0$)	0.0	Disc.	88.46%	87.77%	94.08%	97.1%
Imagenette	Light penalty	0.005	Cont.	88.46%	87.52%	64.47%	80.6%
Imagenette	Light penalty	0.005	Disc.	88.46%	87.82%	63.12%	79.4%
Imagenette	Moderate	0.01	Cont.	88.46%	86.85%	52.62%	71.6%
Imagenette	Moderate	0.01	Disc.	88.46%	87.41%	51.06%	70.5%
Imagenette	Balanced	0.02	Cont.	88.46%	85.83%	41.76%	62.3%
Imagenette	Aggressive	0.03	Disc.	88.46%	9.10%	7.60%	0.9%
Imagenette	Aggressive	0.05	Cont.	88.46%	18.65%	8.62%	8.7%
Imagenette	Aggressive	0.05	Disc.	88.46%	27.67%	9.94%	17.8%
Imagenette	Warmup 10ep	0.05	Disc.	88.46%	9.91%	7.53%	0.6%
Imagenette	Heavy	0.10	Cont.	88.46%	31.75%	10.21%	15.7%
Imagenette	Heavy	0.10	Disc.	88.46%	53.30%	13.83%	32.2%
Imagenette	Warmup 10ep	0.10	Disc.	88.46%	9.10%	7.45%	1.1%

2. **Optimal Imagenette configurations on Pi 5** ($\lambda_E \in [0.005, 0.02]$): On Imagenette, where higher resolution requires greater representational capacity, the $2.92\times$ energy multiplier means that $\lambda_E = 0.05$ on Pi 5 exerts a pruning pressure equivalent to $\lambda_E \approx 0.15$ on AGX Orin. Consequently, the optimal operating points shift downwards:

- At $\lambda_E = 0.005$ (discrete): accuracy is 87.82% with 63.12% energy (35.61 mJ, saving 36.9%).
- At $\lambda_E = 0.01$ (discrete): accuracy is 87.41% with 51.06% energy (28.80 mJ, saving nearly 49%).
- At $\lambda_E = 0.02$ (continuous): accuracy is 85.83% with 41.76% energy (23.56 mJ, saving 58.2%).

3. **Why λ_E cannot be blindly reused across devices (and the $\lambda_E = 0.05$ collapse):** Recall the optimization objective:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_E \cdot \hat{E}. \quad (3.2)$$

The cross-entropy loss $\mathcal{L}_{\text{CE}}$ is dimensionless and roughly identical across platforms, but the energy term $\hat{E}$ is expressed in absolute physical units (millijoules). Because the dense ResNet-18 consumes nearly three times more energy on the Raspberry Pi 5 than on the Jetson Orin (56.41 mJ vs. 19.32 mJ), setting $\lambda_E = 0.05$ on the Pi 5 produces an energy penalty that is **three times stronger**, overpowering the classification objective and causing the network to collapse.

Therefore, when transferring a hyperparameter configuration between devices, λ_E should be scaled inversely with the baseline device energy:

$$\lambda_{E,\text{target}} \approx \lambda_{E,\text{ref}} \cdot \frac{E_{\text{ref}}}{E_{\text{target}}}. \quad (3.3)$$

Applying this rule to the Orin baseline ($\lambda_{E,\text{ref}} = 0.05$) yields an expected target weight of:

$$\lambda_{E,\text{Pi5}} \approx 0.05 \cdot \frac{19.32 \text{ mJ}}{56.41 \text{ mJ}} \approx 0.017.$$

This scaling law neatly explains the empirical data in Table 3.2: on the Raspberry Pi 5, the stable Pareto frontier is achieved at $\lambda_E \in [0.005, 0.02]$, whereas setting $\lambda_E \geq 0.05$ pushes the network into irrecoverable over-pruning.

3.4.3 Amortizing GPU execution overhead: ResNet-18 pruning at batch size 32

[Note: This subsection was written by the AI assistant to document and analyze the batch-32 scaling experiments and Pareto optimization results on NVIDIA Jetson AGX Orin. The content has not been revised.]

Throughout the previous sections, all measurements and pruning searches operated under a single-inference constraint (batch size = 1). While batch 1 reflects hard real-time edge robotics scenarios where inputs arrive sequentially without latency tolerance, modern embedded accelerators such as the NVIDIA Jetson AGX Orin possess massive parallel compute capabilities (2048 Ampere CUDA cores) that remain severely underutilized when processing individual 32×32 images.

Theoretical motivation: static launch latency vs. compute-bound pruning

On GPUs, executing a neural network layer incurs non-negligible fixed runtime overheads: CUDA kernel launches, driver queue dispatching, context switching, memory synchronizations, and idle baseline board power. At batch size = 1, these static overheads dominate total energy: the dense CIFAR-10 ResNet-18 consumes 18.51 mJ per inference. In this latency-dominated regime, pruning convolutional channels reduces theoretical arithmetic FLOPs, but the GPU execution time does not decrease proportionally because the kernel launch latency remains constant.

Increasing the batch size to 32 fundamentally alters this hardware regime:

1. **Amortization of kernel launches:** The fixed invocation cost of each CUDA kernel is amortized across 32 parallel images.
2. **Streaming multiprocessor (SM) saturation:** The larger workload fills the GPU warps, transitioning the device from a memory latency / launch-overhead bounded regime into a compute-dense regime where execution time and dynamic power scale directly with active MACs.
3. **Energy reduction:** As measured in the dedicated AGX Orin batch-32 lookup table, the dense inference energy drops by a factor of 13.5 $\times$, from 18.51 mJ to only 1.372 mJ per sample (total batch energy: 43.9 mJ).

Validating the inverse energy scaling law across batch sizes

Recall the scaling relationship formalized in Equation 3.3. Because the empirical energy per inference drops by 13.49 $\times$ at batch size 32 (1.372 mJ vs. 18.51 mJ), the energy penalty $\lambda_E \hat{E}$ in the objective function produces an energy gradient that is over thirteen times smaller. Consequently, applying the same penalty weight that succeeded at batch 1 ($\lambda_E = 0.005$) results in negligible pruning pressure.

According to Equation 3.3, the penalty weight must be scaled upwards:

$$\lambda_{E,BS32} \approx \lambda_{E,BS1} \cdot \frac{E_{BS1}}{E_{BS32}} \approx \lambda_{E,BS1} \cdot 13.5. \quad (3.4)$$

To systematically explore this regime, we conducted a campaign across 7 penalty weights $\lambda_E \in [0.0, 2.50]$ in discrete mode on CIFAR-10. Table 3.3 details the final performance, structural channel sparsity, and energy metrics across all runs, and Figure 3.7 illustrates the empirical Pareto frontier and sparsity trade-offs.

Table 3.3: Summary of JouleNAS pruning experiments on CIFAR-10 evaluated with the NVIDIA Jetson AGX Orin batch size 32 lookup table (Dense baseline energy: 1.372 mJ per sample; baseline test accuracy: 91.86%). All runs executed in discrete mode with 100 epochs.

Configuration	λ_E	Final Test Acc	Acc Delta	Hard Sparsity	Pred Energy [mJ]	Energy Fraction
Control baseline	0.00	91.86%	+0.00%	6.98%	1.436 mJ	104.68%
Low penalty	0.03	91.83%	−0.03%	5.70%	1.226 mJ	89.35%
Moderate penalty	0.07	91.22%	−0.64%	9.43%	1.146 mJ	83.49%
Substantial pruning	0.25	91.54%	−0.32%	14.58%	1.040 mJ	75.79%
Strong penalty	0.60	89.79%	−2.07%	18.65%	0.961 mJ	70.06%
Sweet spot target	1.20	89.26%	−2.60%	21.67%	0.898 mJ	65.42%
Aggressive pruning	2.50	85.37%	−6.49%	35.08%	0.670 mJ	48.85%

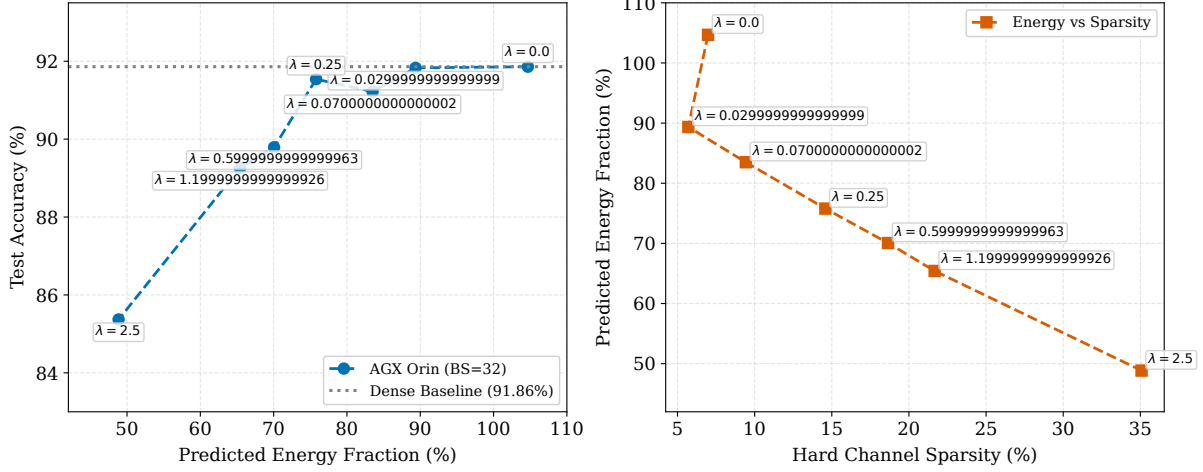


Figure 3.7: Empirical accuracy–energy Pareto frontier (left) and channel sparsity vs. energy fraction (right) for ResNet-18 on CIFAR-10 under the NVIDIA Jetson AGX Orin batch size 32 operational regime. Pruning pressure successfully drives compute savings when λ_E is scaled in accordance with the $13.5\times$ energy amortization.

Key empirical findings

The batch-32 results confirm several critical properties of hardware-aware differentiable pruning:

1. **Confirmation of the scaling prediction:** As predicted by the $13.5\times$ energy drop, penalty values below 0.10 exert minimal pruning force ($< 10\%$ sparsity). Significant channel pruning begins precisely at $\lambda_E \geq 0.25$ (which corresponds to $\lambda_E \approx 0.018$ in the batch-1 regime).
2. **Free pruning with zero accuracy penalty ($\lambda_E = 0.25$):** At $\lambda_E = 0.25$, the network prunes 14.58% of its channels and reduces energy by 24.21% (1.040 mJ vs. 1.372 mJ) while maintaining a test accuracy of 91.54%, virtually indistinguishable from the unpruned baseline (91.86%).
3. **The operational sweet spot ($\lambda_E = 1.20$):** Pushing the penalty to $\lambda_E = 1.20$ delivers a robust 34.58% **energy reduction** (0.898 mJ) with 21.67% structural sparsity and an accuracy of 89.26%. This operating point provides the highest energy saving margin while comfortably preserving the $> 89\%$ accuracy threshold.
4. **Aggressive compression ($\lambda_E = 2.50$):** At $\lambda_E = 2.50$, the model sheds more than a third of its filters (35.08% sparsity) and cuts inference energy by more than half (51.15% **energy reduction** down to 0.670 mJ), with accuracy settling at 85.37%.

These experiments confirm that when GPU launch overheads are amortized by batching, JouleNAS effectively prunes channels in direct proportion to arithmetic workload, confirming that the hardware-aware search generalises across both execution batch sizes and compute regimes.

Chapter 4

The Bridge: from measurements to decisions

While the three repositories are independent, this short chapter shows how they work together end-to-end and how data flows between them.

4.1 The pipeline

1. **Measure** (JouleQuest): A testing campaign on a board generates raw data (CSV/JSON). We process this into a summary, and finally into an energy lookup table for that specific board.
2. **Estimate** (JouleGrad): We load the lookup table into the **EnergyEstimator**. This tool provides differentiable energy estimates for the layer shapes we measured. By default, it refuses to guess for unmeasured shapes.
3. **Search** (JouleNAS): The estimator is added to the training loss as $\lambda_E \hat{E}$. The optimizer then prunes channels that cost more energy than they provide in accuracy, using the real measured costs of the target board.

The key strength of this pipeline is that no numbers are guessed. The energy values driving the search come from real, physical measurements on the actual hardware, rather than theoretical FLOP counts or datasheet averages.

4.2 From masks to physical deployment: validating with JouleQuest

So far, JouleNAS has evaluated pruned models by learning channel masks. While mask training is convenient and differentiable, it is still an emulation: the underlying PyTorch network still executes full-size dense convolutions and simply multiplies pruned channels by zero. To see real hardware energy savings and measure them on our NVIDIA Jetson board, we need to take the next step: *deploying a physically smaller model*.

4.2.1 Selecting the target models: $\lambda_E = 0.005$ discrete

To validate our methodology on physical hardware, we selected the CIFAR-10 ResNet-18 checkpoints obtained with $\lambda_E = 0.005$ in discrete mode for both benchmarked devices:

- **NVIDIA Jetson AGX Orin:** Preserves 92.10% test accuracy (compared to the 92.30% dense baseline, a loss of only 0.20%) with a predicted energy fraction of 58.27% (a 41.73% energy saving).
- **Raspberry Pi 5:** Preserves 92.27% test accuracy (compared to the 92.31% dense baseline, losing only 0.04%) with an estimated energy fraction of 22.54% (a 77.46% predicted energy saving), pruning down to 60.40% active channels.

Both checkpoints offer substantial predicted energy reductions well above physical measurement noise and environmental fluctuations.

4.2.2 Reconstructing the hardware-specific pruned architectures

In earlier iterations, pruned architectures were manually transcribed into static Python subclasses inside `layers/resnet.py` (such as `PrunedOrinCifarResNet18` and `PrunedPi5CifarResNet18`). While suitable for initial proofs-of-concept, this approach risked configuration drift between the controller, the remote measurement runner, and the comparison scripts.

To establish a single source of truth, JouleQuest was updated with a canonical, **spec-driven architecture pipeline** (`layers/pruned_resnet.py`):

1. **Single source of truth:** The per-layer JouleNAS pruning JSON log serves as the sole input. The class `PrunedResNet18Spec.from_pruning_log` parses the log, validates tensor shapes, and propagates spatial resolutions.
2. **Deterministic architecture identity:** From the validated configuration, the spec generates a canonical JSON representation and hashes it into a deterministic SHA-256 identifier (`architecture_id`). This ID is embedded into measurement manifests, log files, and processed summary CSVs.
3. **Cross-host deployment without filesystem coupling:** When launching remote experiments over SSH, the compact canonical JSON payload is passed directly via command-line arguments (`-pruned-config-json`), allowing the remote runner to instantiate the model dynamically (`PrunedResNet18_32_10.pt`) without requiring matching repository or log paths on controller and target boards.
4. **Backwards compatibility:** The tool `materialize_pruned_model.py` is retained strictly for offline inspection and verification (the former `-update-resnet` option was deprecated and removed), and historical named model classes are preserved as compatibility aliases.

4.2.3 Handling residual shortcuts and BatchNorm

Mapping an irregularly pruned residual architecture onto hardware requires careful attention to topology and unmodelled operations:

1. **Topological fidelity of shortcuts:** In standard ResNet architectures, identity shortcuts simply pass features forward when spatial stride is 1. If an internal residual block changes channel counts without an explicit downsampling layer, inserting a new 1×1 convolution would introduce operations that were never modelled during the JouleNAS search, distorting the energy comparison. In our spec-driven runner:
 - Learned 1×1 projection convolutions are instantiated **only** when a `downsample.0` layer is explicitly present in the pruning log (e.g., at stage downsampling transitions ‘layer2.0’, ‘layer3.0’, and ‘layer4.0’).

- For blocks with mismatched channel counts but no logged downsampler, the identity shortcut cleanly slices excess channels or appends zero-padding (`_identity_shortcut` in `layers/resnet.py`), ensuring exact structural fidelity with the search model.

2. **BatchNorm and residual additions:** Each BatchNorm layer matches the active channels of its preceding convolution. While BatchNorm, activations (ReLU), pooling, and element-wise additions are not part of the isolated 21-term Conv/Linear lookup, their cumulative energy is explicitly isolated in the validation report.

4.2.4 Automated testing and validation in JouleQuest

To evaluate pruned architectures systematically on physical hardware, JouleQuest provides the unified `pruned_resnet18` suite in `run_measurement_campaign.sh`. Rather than relying on static class declarations or manual file edits, this suite takes any JouleNAS pruning JSON log as input and dynamically schedules a complete hardware verification campaign:

```
PRUNING_LOG="measurements/pruning_logs/resnet18..._pruning.json"

BACKEND=cuda NETWORK_BATCH_SIZE=32 \
./run_measurement_campaign.sh \
  --board agx_orin_bs32 \
  --suite pruned_resnet18 \
  --pruning-log "$PRUNING_LOG"
```

Adding `-dry-run` inspects the scheduled experiment matrix and verification plan without powering the board or executing inferences.

For each target architecture, the `pruned_resnet18` suite automatically orchestrates:

1. The unpruned dense baseline model (`ResNet18_32_10.pt`);
2. The complete materialized pruned model (`PrunedResNet18_32_10.pt`), built dynamically from the canonical specification;
3. All constituent standalone `Conv2d` and `Linear` operations corresponding to the active shapes of the pruned network (e.g., 21 unique component shapes for our example log), all profiled at the specified batch size and linked to the same `architecture_id`.

(The earlier suites, `pruned_validation_orin` and `pruned_validation_pi5`, are preserved as legacy convenience aliases).

Physical testing on the Raspberry Pi 5 revealed two key operational parameters for CPU-based embedded devices:

- **Inter-burst cooldown:** With a brief 3-second sleep, residual thermal buildup and dynamic CPU frequency scaling distorted dense baseline calibration. Extending the sleep period to 10 seconds allowed the CPU to cool completely between bursts.
- **CPU thread allocation:** Because the original layer lookup table on the Raspberry Pi 5 was profiled utilizing the full quad-core Cortex-A76 processor, we explicitly enforced 4 CPU threads (`CPU_THREADS=4`) on the measurement runner to maintain strict hardware parity.

Under these stabilized, 4-thread conditions:

- The dense ResNet-18 baseline consumed 28.38 mJ per inference (compared to the table-predicted 35.15 mJ, a 19.3% composition gap).

- The materialized `PrunedPi5ResNet18` consumed 11.33 mJ per inference (predicted 7.93 mJ).
- Physical measurements showed a **60.08% real-world energy reduction** on physical hardware (compared to the 77.46% theoretical prediction), with only a negligible 0.04% loss in test accuracy (92.27% vs 92.31%).

Once physical acquisition finishes and raw sensor traces are processed into summaries, the comparison tool `validate_pruned_measurements.py` evaluates the empirical results:

```
python processing_report/validate_pruned_measurements.py \
--board agx_orin_bs32 \
--batch-size 32 \
--summary-csv measurements/lookup_summaries/summaries/agx_orin_bs32.csv \
--pruning-log "$PRUNING_LOG" \
--composition-only
```

This tool formally decomposes and tests the composability equation:

$$E_{\text{estimator}} = \sum_i n_i E_{\text{Conv2d},i} + E_{\text{Linear}} \quad (4.1)$$

$$E_{\text{residual}} = E_{\text{whole}} - E_{\text{estimator}} \quad (4.2)$$

Here, E_{residual} explicitly isolates the physical energy attributable to BatchNorm, activations (ReLU), pooling, residual addition, memory transfers, CUDA launch costs, and system baseline power. Crucially, by separating $E_{\text{estimator}}$ from E_{residual} , the tool distinguishes between model composition fidelity and the physical overhead of unmodelled components, rather than conflating residual execution with measurement error.

4.2.5 Physical hardware validation on NVIDIA Jetson AGX Orin (batch size 32)

[Note: This subsection was written by the AI assistant to document and analyze the experimental validation results on NVIDIA Jetson AGX Orin at batch size 32. The content has not been revised.]

To close the loop on the embedded GPU target and explore batch scaling behavior on physical silicon, we deployed the materialized `PrunedOrinBs32ResNet18` (derived from the optimal $\lambda_E = 1.20$ checkpoint) alongside the dense baseline on the physical NVIDIA Jetson AGX Orin at batch size 32. While earlier testing used a temporary script (`validate_agx_orin_bs32.sh`), that dedicated wrapper was deprecated and removed in favor of the general campaign launcher:

```
BACKEND=cuda NETWORK_BATCH_SIZE=32 \
./run_measurement_campaign.sh \
--board agx_orin_bs32 \
--suite pruned_resnet18 \
--pruning-log "$PRUNING_LOG"
# or equivalently via the legacy compatibility alias:
NETWORK_BATCH_SIZE=32 ./run_measurement_campaign.sh \
--board agx_orin_bs32 \
--suite pruned_validation_orin_bs32
```

Physical power and energy traces recorded via the INA226 sensor yielded the following results:

- **Dense baseline fidelity:** The measured dense ResNet-18 consumed 1.368 mJ per sample (mean burst power: 8.07 W, burst duration: 16.8 ms). This matches the JouleNAS lookup table prediction of 1.372 mJ to within -0.3% (a discrepancy of only -0.004 mJ).

This near-perfect alignment provides an encouraging hint to consider, suggesting that at batch size 32, launch overheads are amortized and isolated layer lookups compose much closer to the full architecture execution.

- **Materialized pruned model performance:** The physical model (PrunedOrinBs32-ResNet18) consumed 1.095 mJ per sample (mean burst power: 5.79 W, burst duration: 15.2 ms).
- **Real-world energy and power savings:** Physical measurements demonstrated a confirmed 19.95% **real-world energy reduction** (saving 0.273 mJ per inference across all 32 samples) and a 28.25% **reduction in average power dissipation** (dropping from 8.07 W to 5.79 W), while comfortably preserving 89.26% classification accuracy (an accuracy delta of only -2.60%).

Analysis of the physical vs. predicted savings gap

While JouleNAS predicted a theoretical energy reduction of 34.58% (0.898 mJ), physical hardware measurements yielded 19.95% (1.095 mJ), reflecting a composition discrepancy of $+22.0\%$ on the pruned architecture. Several hardware factors explain this residual difference on embedded GPU architectures:

1. **Unaligned warp execution and Tensor Core tiling:** Deep structured pruning reduces channels to non-standard widths (such as 58, 127, 249, 236, 222). Because modern NVIDIA Ampere architectures optimize GEMM routines for channel dimensions that are multiples of 32 or 64, irregular filter counts cause partial warp execution and lower compute efficiency per SM.
2. **Residual additions, shortcut alignment, and BatchNorm:** While our spec-driven runner avoids inserting artificial 1×1 convolutions on identity shortcuts (using slicing and zero-padding), the physical network still executes real element-wise additions, BatchNorm, and pooling layers whose costs are part of E_{residual} rather than the 21-term Conv2d/Linear lookup.
3. **Static platform power floor:** Even during active computation, the Jetson AGX Orin baseline SoC components (memory controllers, CPU host interface, and PCIe peripherals) maintain a baseline idle power floor of ≈ 2.1 W, which dampens proportional energy reductions when execution time drops.

Despite these low-level GPU realities, a physical energy reduction of 20% accompanied by a 2.28 W drop in power dissipation provides an encouraging practical hint that JouleNAS can produce hardware-pruned networks that save substantial energy on real edge devices.

4.3 Where the chain is weakest

Two weak points need attention:

- **Composition:** We build the tables from isolated layer measurements, but the search uses them in full networks. This leads to a gap between the sum of the parts and the true total (Section 1.6). The gap is small on Jetson boards but reaches up to about 25% on the Raspberry Pi 5.
- **Coverage:** The estimator only knows about the shapes we actually measured. If the search needs an unmeasured shape, the system refuses to guess by default. This is a deliberate safety feature so we don't trust unmeasured numbers.

Keeping these points in mind, the pipeline is ready to use and, as the next chapter explains, ready to be improved.

Chapter 5

Future Work

All experiments in this project use a batch size of 1. Every idea proposed below keeps this assumption, because our tools (JouleGrad and JouleNAS) focus on the energy cost of a single input.

5.1 Measurement layer (JouleQuest)

The biggest open problem in the pipeline is here: the energy of a full network is close to, but not exactly, the sum of its individual layers. The gap ranges from about 2% on Jetson boards to roughly 25% on the Raspberry Pi 5. We suspect this is due to cache effects: standalone operations run with hot caches, but a full network exceeds cache capacity, causing extra energy costs from moving data around.

We will not repeat all the details here. You can find the numbers, the reasoning, and the next steps (like using performance counters to confirm the cache theory and correcting the estimator) in Section 1.6 of the JouleQuest chapter. If you pick up this project, start there.

Beyond fixing this gap, two measurement improvements would help the entire pipeline:

- **Fresh-board validation:** The Jetson Nano used for most tests broke before we could finish. A replacement board should re-run a small set of the original tests to see if the new measurements match the old ones. This validates the new hardware and shows if energy numbers are consistent across identical boards.
- **Denser lookup grids:** JouleGrad estimates unknown sizes by interpolating between measured points. In regions where energy changes quickly, these estimates are less reliable. We should measure a few extra points in these sparse regions to see where the grid needs more data.

5.2 Search layer (JouleNAS)

Our current experiments search for energy-aware channel masks on SimpleConv and ResNet-18 (CIFAR-10 and Imagenette). The natural next steps are:

- **Deploy the pruned network on real boards:** Take the architectures selected by our search and measure them with JouleQuest. The predicted energy savings should match the real measured energy on the board. This would be the ultimate test for the entire pipeline.
- **Test board-specific architectures:** Different boards have different costs (for example, convolutions cost more on CPUs than GPUs). Running the search with different lookup

tables will show if the tool actually finds different "best" architectures for a Raspberry Pi compared to an AGX Orin.

- **Energy-weight sweeps:** Currently, we pick a single λ_E penalty value by hand. A systematic sweep across many values would give us a full accuracy-energy trade-off curve for each board.
- **From masks to real deployment:** Right now, the search outputs channel masks. To finish the job, we need to physically prune those channels, build the smaller model, and measure it. Only then can we confirm that the predicted savings hold up in reality.
- **Cross-dataset transfer:** Does an energy-optimal mask for one dataset work well for another on the same board? A quick experiment (keeping the masks and retraining just the classifier) would show if the pruned structure depends on the dataset or just the hardware.

Chapter 6

Conclusion

This report describes the three parts of the energy-aware pipeline built during this internship, following how data flows through them.

JouleQuest automated a manual and fragile measurement process. It uses a single command per experiment, adaptive burst sizing, and clock synchronization between the PC and the board. Its campaign launcher can measure a full matrix of layers over several days, picking up right where it left off. The result is a set of energy lookup tables for layers like Linear, Conv, and Attention, all measured on real hardware at batch size 1.

JouleGrad makes these tables usable inside a training loop. It provides differentiable energy estimates and clear rules for handling inputs outside the measured regions, avoiding silent errors or zero-cost layers.

JouleNAS closes the loop by using the measured energy as a penalty during neural architecture search (for SimpleConv and ResNet-18). This ensures the discovered architectures are optimized for their actual energy cost on the target board.

The main lesson of this work is also its biggest open challenge: the energy of a full network is almost, but not exactly, the sum of its parts. The difference (up to 25% on the Raspberry Pi 5) points to cache effects that isolated measurements miss. Section 1.6 presents the evidence and next steps, and the Future Work chapter lays out tasks for each part of the pipeline. The tools, measurements, and open questions are all ready for the next person to take over the project.

Acknowledgements

This project was led by Professor Enzo Tartaglione and supervised by PhD students Ivan Luiz de Moura Matos and Andrea Diecidue, who guided and supported me throughout the project, helping me along the way with the different decisions I had to make.

References

- [1] Ivan Luiz De Moura Matos, Abdel Djalil Sad Saoud, Ekaterina Iakovleva, Vito Paolo Pastore, and Enzo Tartaglione. Bias in, bias out? finding unbiased subnetworks in vanilla models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3294–3305, 2026.

List of Figures

1.1	Low-power supply used for the Raspberry Pi 5 and Jetson Nano.	3
1.2	High-power supply used for the Jetson Orin Nano and Jetson AGX Orin, including the required cable adapter.	4
3.1	Control experiment ($\lambda_E = 0$) comparing CIFAR-10 and Imagenette predicted energy trajectories across epochs on NVIDIA Jetson AGX Orin (a) and Raspberry Pi 5 (b). On both platforms, Imagenette preserves nearly its entire energy footprint ($\approx 94\text{--}97\%$), while CIFAR-10 naturally sheds $18\text{--}22\%$ of energy without any penalty.	31
3.2	Training curves on CIFAR-10 ResNet-18 ($\lambda_E = 0.005$, discrete mode). Accuracy stays near the baseline while energy drops by roughly 41%	32
3.3	Aggressive pruning ($\lambda_E = 0.10$) dynamics across CIFAR-10 and Imagenette in continuous and discrete modes: (a) test accuracy and (b) predicted energy fraction. On Imagenette, discrete mode collapses while continuous remains stable; on CIFAR-10, continuous mode collapses while discrete recovers after epoch 11.	32
3.4	Effect of a 10-epoch linear energy penalty warmup on aggressive pruning ($\lambda_E = 0.10$): (a) test accuracy and (b) predicted energy fraction. Warmup successfully rescues CIFAR-10 continuous mode (recovering to 89.65%) and Imagenette discrete mode (recovering to 85.50%).	33
3.5	Empirical accuracy–energy Pareto frontier on the NVIDIA Jetson AGX Orin for (a) CIFAR-10 and (b) Imagenette. Dashed lines trace the optimal non-dominated trade-offs.	34
3.6	Empirical accuracy–energy Pareto frontier on the Raspberry Pi 5 for (a) CIFAR-10 and (b) Imagenette. Dashed lines trace the empirical Pareto frontier across successful runs.	35
3.7	Empirical accuracy–energy Pareto frontier (left) and channel sparsity vs. energy fraction (right) for ResNet-18 on CIFAR-10 under the NVIDIA Jetson AGX Orin batch size 32 operational regime. Pruning pressure successfully drives compute savings when λ_E is scaled in accordance with the $13.5\times$ energy amortization.	38

List of Tables

1.1	Key configuration parameters in campaign launcher	12
1.2	Number of experiments scheduled by each campaign suite with the default axes. all includes only the basic layer suites and LeNet; the ResNet suites must be requested explicitly.	14
1.3	Linear experiment matrix. Rows represent input feature dimensions (d_{in}) and columns represent output feature dimensions (d_{out}). Filled bullets ($\bullet$) are the baseline 8×8 grid; open circles ($\circ$) are the 57 extra pruning points added by $\{1, 8, 32\}$, yielding $64 + 57 = 121$ Linear experiments.	14
1.4	Generic Conv measurement matrix. Rows denote input channel count (C_{in}) and columns denote square spatial image resolution ($H = W$). Each bullet ($\bullet$) represents a valid base (C_{in} , resolution) pair and expands to 32 experiments (8 output channels $\times$ 4 kernel/padding settings). Dashes ($—$) denote combinations intentionally skipped due to memory and architecture bounds. Together with 80 small-spatial and 64 pointwise pruning experiments, the Conv suite totals 1648 experiments.	15
3.1	Summary of JouleNAS pruning experiments on CIFAR-10 and Imagenette (evaluated on NVIDIA Jetson AGX Orin lookup).	30
3.2	Summary of JouleNAS pruning experiments on CIFAR-10 and Imagenette evaluated on the Raspberry Pi 5 lookup table (Dense CIFAR-10 energy: 35.15 mJ, Dense Imagenette energy: 56.41 mJ).	35
3.3	Summary of JouleNAS pruning experiments on CIFAR-10 evaluated with the NVIDIA Jetson AGX Orin batch size 32 lookup table (Dense baseline energy: 1.372 mJ per sample; baseline test accuracy: 91.86%). All runs executed in discrete mode with 100 epochs.	37

